Dédicaces

À mon très cher père,

Tu as toujours été un homme de bien, généreux et juste, donnant sans jamais rien attendre en retour. Tu t'es épuisé, tu as travaillé dur, sans jamais te plaindre, simplement pour m'offrir une vie meilleure, pour me voir réussir. Ton amour inconditionnel, ta présence rassurante, et tes sacrifices silencieux restent à jamais gravés en moi. Même si ton absence me fait terriblement mal, je sens encore ta force et ta bienveillance m'accompagner chaque jour. Ce projet, je te le dédie de tout mon cœur. Tu me manques infiniment, mon cher papa.

À ma chère maman,

Ta douceur, ta patience et ton amour infini ont été ma lumière dans l'obscurité. Tu as cru en moi, tu as été mon pilier, ma force, mon refuge. Merci pour ta présence, ton écoute, tes prières, et ton amour inépuisable. Ce projet est aussi le fruit de ton courage et de ton cœur. Je t'aime profondément, maman.

À Rym, Salah, Ghofrane, Abdrahmen, Omar et Zayno,

Je ne trouverai jamais les mots pour exprimer toute ma gratitude. Vous êtes mon refuge, mon équilibre, mon essentiel. Merci d'être toujours là, dans le silence ou dans le tumulte. Merci pour votre amour et votre présence. Je n'imagine pas ma vie sans vous.

À Mes chers oncles,

merci pour votre soutien sincère, pour votre bienveillance et pour l'amour . Je suis profondément reconnaissante pour tout ce que vous avez fait pour moi.

À Malak, Mariem, Rayhanne, Houssem, Sarra, Maryem et Ameni,

Mes amies de cœur, merci d'être là, d'écouter mes silences et de comprendre mes tempêtes. Merci de m'encourager, de croire en moi, de partager mes peines comme mes joies. Vous êtes des âmes précieuses dans ma vie. Merci de tout mon cœur.

À Anis, Maroua et Asma,

Mes grands amis, je vous aime sincèrement. Merci pour vos mots d'encouragement, votre soutien sans faille, vos précieux conseils. Merci de m'avoir aidée à traverser les moments difficiles, de m'avoir donné la force de continuer, d'aimer, de rêver, et de croire encore en la vie et en mes rêves.

Et enfin, à moi-même — à Ayouta,

Je veux te féliciter. Pour avoir tenu bon. Pour avoir continué à croire, même dans l'obscurité. Pour ne pas avoir abandonné malgré les déceptions, malgré les chutes. Chaque difficulté t'a appris quelque chose. Chaque pas t'a menée un peu plus loin. Continue de rêver. Continue de croire en toi, en tes valeurs, en Dieu. Reste présente à toi-même, reste consciente.

Remerciements

C'est un grand plaisir et un moment très agréable de rendre hommage et de formuler mes remerciements aux personnes qui, d'une manière ou d'une autre, ont apporté leur soutien et contribué à finaliser ce travail.

Je tiens à exprimer mes vifs remerciements et à témoigner toute ma reconnaissance à Monsieur BENNACEUR Hatem, mon encadrant à l'entreprise d'accueil MOBELITE, pour m'avoir accueillie au sein de l'entreprise et pour m'avoir encadré et orienté durant mon PFE.

J'adresse également mes sincères remerciements et mon profond respect à Monsieur HAMEL Lazhar, mon encadrant à Institut Supérieur d'Informatique et de Mathématiques de Monastir pour son encouragement, ses conseils, son aide et son soutien qu'il m'a apportés pour la réalisation de ce projet. J'ai apprécié sa précieuse disponibilité et sa discrétion. Je lui suis également reconnaissant pour toutes les informations qu'il m'a transmises.

Enfin, je tiens à remercier les membres du jury qui ont accepté de juger mon travail. J'associe à ces remerciements tous les enseignants de Institut Supérieur d'Informatique et de Mathématiques de Monastir qui ont contribué à ma formation. En espérant que je serai toujours à la hauteur et que vous trouverez dans ce travail le fruit de vos conseils et soutiens.

Aya Jrah

Résumé

Ce rapport présente le développement et la mise en œuvre d'un système automatisé d'analyse de sécurité pour des dépôts de code, intégrant plusieurs outils open source (SonarCloud, Trivy, Snyk, OWASP Dependency Check) au sein d'une architecture modulaire. À partir des objectifs du projet, nous explorons les solutions existantes, les exigences fonctionnelles et techniques, ainsi que la liste complète des fonctionnalités attendues. L'étude conceptuelle détaille l'architecture logicielle, la conception des modules principaux (gestion des utilisateurs, orchestration des scans, restitution des résultats), en mettant l'accent sur la modularité, la sécurité et les bonnes pratiques de développement. La phase de mise en œuvre décrit les technologies, outils et processus adoptés, notamment l'intégration de scripts d'automatisation, l'utilisation d'APIs REST, la gestion centralisée des résultats, ainsi que l'utilisation de GitHub pour l'hébergement du code source, de GitHub Actions pour l'automatisation des workflows de scan, et de Docker pour l'isolation et le déploiement des différents outils d'analyse. L'automatisation des analyses de sécurité répond aux objectifs d'efficacité, de fiabilité et de scalabilité, tout en facilitant la gestion et la supervision des vulnérabilités détectées. Ce système permet d'améliorer la sécurité des projets analysés, d'accélérer les cycles de détection et de correction, et d'offrir une base solide pour l'évolution vers de nouvelles pratiques d'automatisation de la sécurité. Dans l'ensemble, ce projet apporte une contribution significative à la gestion de la sécurité applicative et à l'automatisation des processus de contrôle qualité dans le développement logiciel .

Mots clés : sécurité, SonarCloud, Trivy, Snyk, OWASP Dependency Check, intégration, déploiement, GitHub, GitHub Actions, Docker, automatisation

Abstract

This report presents the development and implementation of an automated security analysis system for code repositories, integrating several open source tools (SonarCloud, Trivy, Snyk, OWASP Dependency Check) within a modular architecture. Based on the project objectives, we explore existing solutions, functional and technical requirements, and the complete list of expected features. The conceptual study details the software architecture and the design of the main modules (user management, scan orchestration, results reporting), with an emphasis on modularity, security, and best development practices. The implementation phase describes the technologies, tools, and processes adopted, including the integration of automation scripts, the use of REST APIs, centralized results management, as well as the use of GitHub for source code hosting, GitHub Actions for automating scan workflows, and Docker for isolating and deploying the various analysis tools. Automating security scans meets the objectives of efficiency, reliability, and scalability, while facilitating the management and monitoring of detected vulnerabilities. This system improves the security of analyzed projects, accelerates detection and remediation cycles, and provides a solid foundation for evolving new security automation practices. Overall, this project makes a significant contribution to application security management and the automation of quality control processes in software development.

Keywords : security, SonarCloud, Trivy, Snyk, OWASP Dependency Check, Integration, deployment, GitHub, GitHub Actions, Docker, automation

Table des matières

Tables des figures . . . . . **VIII**

Liste des tableaux . . . . . **X**

1 Contexte général du projet **2**

Introduction 4

1.1 Présentation de l'organisme d'accueil 4

1.2 Contexte général de la cybersécurité 5

1.2.1 La sécurité des applications web 5

1.2.2 Menaces courantes : le Top 10 de l'OWASP 6

1.3 Tests de sécurité 7

1.3.1 Méthodes de test 7

1.3.2 Tests manuels vs automatisés 7

1.4 Étude et critique de l'existant 8

1.5 Solution proposée 8

1.6 Intégration de DevOps dans les tests de sécurité 9

1.6.1 Contexte DevOps 9

1.6.2 Intégration continue (CI) et déploiement continu (CD) 9

1.6.3 Surveillance 10

1.6.4 Cycle de vie DevOps 10

1.7 Méthodologie adoptée 11

1.7.1 Choix de la méthode 11

1.7.2 Les méthodes agiles 11

1.7.3 Présentation de la méthodologie Scrum 12

1.7.4 Les rôles de Scrum 12

1.7.5 Les événements de Scrum 12

1.7.6 Les outils de Scrum 13

1.7.7 Les artefacts de Scrum 13

1.7.8 Fonctionnement de Scrum 13

Conclusion 14

2 SPRINT 0 : Etude techniques **15**

Introduction 16

2.1 Spécification des besoins 16

2.1.1 Les besoins fonctionnels 16
2.1.2 Spécification semi-formelle des besoins 17
2.1.3 Besoins non fonctionnels 18
2.1.4 Environnement de développement et choix techniques 19
2.2 Architecture générale de l'application 23
2.3 Conception détaillée 25
2.3.1 Aperçu des classes du projet 25
2.3.2 Diagramme de séquence d'authentification 26
2.3.3 Diagramme de séquence d'inscription 27
2.4 Architecture CI/CD 27
2.4.1 Pipeline CI/CD 28
2.4.2 Mise en place de GitHub 28
Conclusion 29

3 Sprint 1 : CI 30
Introduction 31
3.1 L'integration Continue dans CheckSec 31
3.2 Backlog du Sprint 1 31
3.2.1 User Story 01 : Inscription de l'utilisateur 32
3.2.2 User Story 02 : Authentification d'utilisateur 34
3.2.3 User Story 03 : Accès au Dashboard 36
3.2.4 User Story 04 : Accès au Résultat de scan 38
3.2.5 User Story 05 : Accès aux pages informatives 39
3.2.6 User Story 06 : Intégration et automatisation des tests de sécurité 41
3.3 Docker Build and Push : 47
Conclusion 47

4 Sprint 2 : Dockerisation et déploiement de l'application 48
Introduction 49
4.1 Déploiement Continu 49
4.2 Backlog du sprint 2 49
4.2.1 User Story 07 : Déploiement de CheckSec 50
4.2.2 Configuration Docker 50
4.3 Déploiement avec Render et Railway 52
4.4 Déploiement local avec Docker et Nginx 55
4.4.1 Déploiement local avec Docker et Nginx 55
4.4.2 Déploiement de la partie Frontend 56
4.5 Déploiement de la partie Backend 58

4.6 Orchestration avec Docker Compose 58
4.7 Avantages et limites du déploiement local 59
4.7.1 Avantages 59
4.7.2 Limites 60
Conclusion 60

Bibliographie **62**

Table des figures

1.1 Logo de la société Mobelite 4
1.2 OWASP Top 10 (Source: `https://www.owasptopten.org`) 6
1.3 Logo et icônes de l'OWASP Top 10 7
1.4 Logo de CheckSec 9
1.5 Cycle de vie de DEVOPS 11
1.6 Les concepts fondamentaux de SCRUM 12
1.7 Fonctionnement de SCRUM 13

2.1 Diagramme de cas d'utilisation de CheckSec 18
2.2 Les outils de Frontend 19
2.3 Les outils de Backend 20
2.4 Les outils de Base de données 20
2.5 Les logiciels utilisés 21
2.6 Postman 21
2.7 Les outils de scan de sécurité 22
2.8 Les outils de DevOps 22
2.9 Architecture générale de l'application 23
2.10 Les classes de CheckSec 25
2.11 Diagramme de séquence d'authentification 26
2.12 Diagramme de séquence d'inscription 27
2.13 Architecture CI/CD 28
2.14 Interface du dépôt GitHub 29

3.1 Les phases de CI 31
3.2 Interface d'inscription 32
3.3 Champs manquant 33
3.4 Utilisateur déjà existant 33
3.5 mot de passe à moins de six caractères 34
3.6 Interface d'Authentification 35
3.7 Interface d'Authentification avec erreur d'un email du mouvais format 35
3.8 Interface d'Authentification avec erreur d'un email ou mot de passe incorrect 36
3.9 Interface de Dashboard 37
3.10 En cas d'erreur d'URL 37
3.11 Interface de résultat de scan 38

3.12 En cas d'erreur de scan 39
3.13 Interface de page À propos 39
3.14 Interface de page Nous contacter 40
3.15 Message envoyé avec succès 40
3.16 Erreur lors de l'envoi du message 41
3.17 Logo de GitHub Actions 42
3.18 Schéma des tests unitaires et outils de sécurité dans GitHub Actions 43
3.19 Tableau de bord de SonarCloud 44
3.20 Résultat de Trivy 44
3.21 Résultats de Trivy dans GitHub Actions 45
3.22 Dashboard de Snyk 45
3.23 Résultats de Snyk dans GitHub Actions 45
3.24 Résultats de scan de OWASP DEPENDENCY-CHECK dans GitHub Actions 46
3.25 Résultats de scan de OWASP DEPENDENCY-CHECK 46
3.26 Identifiants Docker stockés dans GitHub Actions 47
3.27 le dashboard de Docker Hub 47

4.1 Pipeline CI/CD 49
4.2 Logo de Docker 50
4.3 Logo de Render 50
4.4 Formulaire de configuration Render 52
4.5 Création de nouveau service web 53
4.6 Formulaire de configuration de service web 1 53
4.7 Formulaire de configuration de service web 2 54
4.8 Connexion de base de données MongoDB Atlas 54
4.9 Configuration et tableau de bord de Railway 55
4.10 Les outils de déploiement local 55
4.11 Dockerfile du frontend 56
4.12 Fichier de configuration Nginx 57
4.13 Entrée dans le fichier `/etc/hosts` 57
4.14 Dockerfile du backend 58
4.15 Fichier `docker-compose.yaml` 59

Liste des tableaux

1.1 Fiche de renseignement de Mobelite 4
1.2 Tableau comparatif entre tests manuels et automatisés 8
1.3 Outils de sécurité existants 8

2.1 Backlog 17
2.2 Résumé des rôles 17
2.3 Besoins non fonctionnels 18
2.4 Spécifications de l'ordinateur 19
2.5 Architecture Frontend 23
2.6 Architecture Backend 24

3.1 Backlog du sprint 1 32

4.1 Backlog du Sprint 2 50

Liste des sigles et acronymes

EIA	Entretien Individuel Annuel
UP	Unified Process
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
JSON	Java Script Objet Notation
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
Sass	Syntactically Awesome Style Sheets
JS	Java Script
SGBD	Système de Gestion de Base de Données
SMTP	Simple Mail Transfer Protocol
MVC	Modal View Controller
MVVM	Modal View View Modal
MCD	Modèle conceptuel de données
UML	Unified Modeling Language

Introduction Générale

Dans un monde où les cybermenaces ne cessent d'évoluer en complexité et en rapidité, la sécurité des applications web est devenue une priorité stratégique pour les entreprises. Selon le rapport annuel de l'ENISA (l'Agence de cybersécurité de l'Union européenne), les attaques contre les applications web ont augmenté de plus de 50 % en 2024. Ce chiffre alarmant souligne l'urgence d'adopter des pratiques de développement plus robustes et sécurisées.

Le génie logiciel, discipline fondamentale de l'informatique, regroupe l'ensemble des méthodes, outils et bonnes pratiques permettant de concevoir, développer, tester et déployer des logiciels de manière structurée et fiable. Le développement d'environnements de développement intégrés (IDE), de systèmes de gestion de versions comme Git, ainsi que de cadres de test, a considérablement facilité le travail des développeurs. Toutefois, l'usage de ces outils ne suffit plus à garantir une sécurité adéquate face aux vulnérabilités potentielles des applications modernes.

Dans ce contexte, l'automatisation des tests de sécurité s'impose comme une réponse nécessaire et pertinente. Elle permet d'intégrer la sécurité dès les premières étapes du cycle de développement, réduisant ainsi les risques et les coûts liés à la découverte tardive de failles.

C'est dans cette optique qu'est né le projet CheckSec, qui vise à automatiser l'analyse de la sécurité des applications web à l'aide d'outils modernes, accessibles via une interface intuitive. Le projet intègre plusieurs outils d'analyse (Snyk, SonarCloud, Trivy, OWASP Dependency-Check) dans une chaîne CI/CD complète. CheckSec ne se limite pas à la détection des vulnérabilités : il génère également des rapports clairs et exploitables, permettant aux développeurs de corriger rapidement les problèmes identifiés. L'objectif est de renforcer la sécurité dès la phase de développement, tout en maintenant un haut niveau de productivité.

Le présent rapport détaille l'ensemble des étapes suivies pour concevoir et mettre en œuvre cette solution :

- Chapitre 1 : Présentation de l'organisation d'accueil, de la méthodologie adoptée et de l'importance de la sécurité des applications web.
- Chapitre 2 : Description du Sprint 0, incluant la planification, la constitution du backlog du projet et le choix des outils.
- Chapitre 3 : Mise en œuvre de l'intégration continue, avec l'intégration des outils de sécurité dans les workflows GitHub Actions.
- Chapitre 4 : Phase de déploiement continu, basée sur Docker et Render.com, garantissant une mise en production fluide et sécurisée.

CHAPITRE 1

Contexte général du projet

Sommaire

Introduction 4

1.1 Présentation de l'organisme d'accueil 4

1.2 Contexte général de la cybersécurité 5

1.2.1 La sécurité des applications web 5

1.2.2 Menaces courantes : le Top 10 de l'OWASP 6

1.3 Tests de sécurité 7

1.3.1 Méthodes de test 7

1.3.2 Tests manuels vs automatisés 7

1.4 Étude et critique de l'existant 8

1.5 Solution proposée 8

1.6 Intégration de DevOps dans les tests de sécurité 9

1.6.1 Contexte DevOps 9

1.6.2 Intégration continue (CI) et déploiement continu (CD) 9

1.6.3 Surveillance 10

1.6.4 Cycle de vie DevOps 10

1.7 Méthodologie adoptée 11

1.7.1 Choix de la méthode 11

1.7.2 Les méthodes agiles 11

1.7.3 Présentation de la méthodologie Scrum 12

1.7.4 Les rôles de Scrum 12
1.7.5 Les événements de Scrum 12
1.7.6 Les outils de Scrum 13
1.7.7 Les artefacts de Scrum 13
1.7.8 Fonctionnement de Scrum 13
Conclusion 14

Introduction

Ce chapitre présente le cadre de notre projet, qui s'inscrit dans un contexte où la cybersécurité est devenue essentielle face à la montée des menaces ciblant les applications web. Nous présentons l'organisme hôte, sa structure et ses services, puis nous exposons l'objectif du projet : intégrer l'automatisation des tests de sécurité dans le cycle de développement. La méthodologie adoptée sera également décrite afin d'expliquer notre approche.

1.1 Présentation de l'organisme d'accueil

Mobelite a été créée en 2012 avec une passion pour les interfaces et le développement produit. Son offre de services repose sur trois piliers : Digital Factory, Conseil, et Solutions. Elle dispose d'équipes commerciales et marketing à Paris, ainsi que d'équipes de développement à Tunis et Monastir.

Fig. 1.1 – Logo de la société Mobelite

Dénomination sociale	Mobelite
Site Web	https://www.mobelite.fr
Secteurs	Services et conseil informatiques
Siège social	38, Rue Eugène Oudiné – 75013 Paris
Fondée en	2012
Directeur de Publication	Mohamed Ltaief
Téléphone	+33 1 83 81 01 20

Tab. 1.1 – Fiche de renseignement de Mobelite

Les services de la société

La gamme de services proposée par Mobelite comprend notamment :

- **Pôle Web :** Ce secteur bénéficie d'une équipe d'experts compétents, engagés dans la fidélisation client. Leur objectif est de concevoir des sites web dynamiques et de développer des applications innovantes centrées sur le web et le commerce électronique.
- **Pôle Mobile :** est caractérisé par la créativité , l'expérience et l'innovation grâce à ses chefs de projets, experts techniques, ingénieurs développeurs et ingénieurs système qui se mobilisent pour

développer des applications dans différents domaines : Android, IOS, Java Mobile et l'optimisation des sites web essentiellement pour les navigateurs mobiles

— **Pôle Test :** Il existe une équipe réactive qui a pour mission le test et la validation des projets et l'assurance de leur bon fonctionnement. « Mobelite » est considérée aussi comme un porteur de projets dans le domaine du développement informatique, elle adopte aussi ses propres solutions comme solution de marketing mobile et solution innovante pour publier des app-books.

— **Pôle IT :** cette équipe est en charge des technologies de l'information et des systèmes informatiques. Il travaille en étroite collaboration avec les équipes de développement pour garantir un déploiement des applications plus rapide, fiable et efficace. Son rôle principal est d'automatiser les processus de développement, de déploiement et de gestion de l'infrastructure IT en utilisant des outils et des méthodes qui favorisent une meilleure communication et collaboration entre les équipes de développement et d'exploitation. L'objectif est d'assurer une livraison continue d'applications de haute qualité, tout en optimisant l'efficacité et la satisfaction des utilisateurs finaux.

1.2 Contexte général de la cybersécurité

Aujourd'hui, avec l'utilisation croissante d'internet et des services en ligne, la cybersécurité est devenue très importante. Que ce soit pour les entreprises, les administrations ou même les particuliers, il est essentiel de protéger les systèmes informatiques contre les attaques. Les applications web sont particulièrement exposées, car elles sont accessibles en ligne à tout moment. Si elles ne sont pas bien sécurisées, elles peuvent contenir des failles que les pirates peuvent facilement exploiter. Ces attaques peuvent avoir de graves conséquences : vol de données, arrêt de service, perte d'argent, ou encore atteinte à la réputation de l'entreprise. Pour éviter cela, il est important de tester régulièrement la sécurité des applications. Mais faire ces tests à la main prend beaucoup de temps, demande des compétences techniques, et peut parfois oublier certains points importants. C'est pour cela que de plus en plus d'organisations choisissent d'automatiser les tests de sécurité. Grâce à cette approche, il devient possible de détecter les failles dès le début du développement, et de les corriger rapidement. Cela permet de gagner du temps, de réduire les erreurs humaines, et d'avoir un meilleur niveau de sécurité. Voici une source reconnue qui confirme bien , venant de l'ANSSI (Agence Nationale de la Sécurité des Systèmes d'Information), qui est une référence officielle en cybersécurité en France https://cyber.gouv.fr/la-cybersecurite-foire-aux-questions.

1.2.1 La sécurité des applications web

La sécurité des applications web est un aspect essentiel du développement. Elle vise à protéger les données des utilisateurs et à prévenir les attaques malveillantes. Une application mal sécurisée peut

être vulnérable à des menaces telles que le vol d'informations, les injections SQL, les vulnérabilités XSS (Cross-Site Scripting) ou les attaques par force brute. Ces vulnérabilités peuvent menacer la confidentialité, l'intégrité et la disponibilité du système. Pour éviter cela, il est crucial d'appliquer de bonnes pratiques de sécurité dès la phase de développement et d'effectuer des tests réguliers pour détecter les vulnérabilités.

1.2.2 Menaces courantes : le Top 10 de l'OWASP

Le Top 10 de l'OWASP est un document de sensibilisation standard pour les développeurs et la sécurité des applications web. Il représente un large consensus sur les risques de sécurité les plus critiques pour les applications web. Il a été lancé en 2003 pour aider les organisations et les développeurs à se doter d'un point de départ pour un développement sécurisé.

Au fil des ans, il s'est transformé en une pseudo-norme qui sert de référence pour la conformité, l'éducation et les outils des fournisseurs. La figure 1.2 représente la liste des Top 10 de l'OWASP

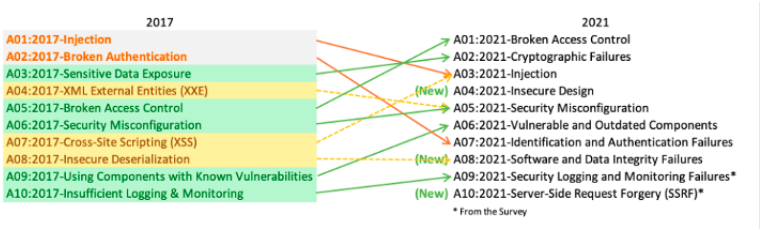

FIG. 1.2 – OWASP Top 10 (Source : `https://www.owasptopten.org`)

- **A01 :2021 – Contrôle d'accès non respecté :** Présente dans 94 % des applications analysées. Cela inclut les failles permettant d'accéder à des ressources sans autorisation.
- **A02 :2021 – Défaillances cryptographiques :** Anciennement "Exposition de données sensibles". Met l'accent sur les erreurs dans l'implémentation de la cryptographie.
- **A03 :2021 – Injections :** Inclut SQL, NoSQL, LDAP, XSS. Toujours très fréquentes, elles concernent 94 % des applications.
- **A04 :2021 – Conception non sécurisée :** Nouvelle catégorie due à une mauvaise conception initiale.
- **A05 :2021 – Mauvaise configuration de sécurité :** Inclut erreurs de configuration, services non protégés, etc.
- **A06 :2021 – Composants vulnérables et obsolètes :** Utilisation de dépendances non à jour ou non sécurisées.
- **A07 :2021 – Échecs d'identification et d'authentification :** Faiblesses dans la gestion des identités et des sessions.

— **A08 :2021 – Défauts d'intégrité des logiciels et des données :** Failles dans les mises à jour logicielles, pipelines CI/CD, etc.

— **A09 :2021 – Défaillances de la journalisation et de la surveillance :** Impact indirect mais critique.

— **A10 :2021 – SSRF :** Nouvelle entrée. Le serveur est forcé à envoyer des requêtes malveillantes.

La figure 1.3 montre les logos et icônes de l'OWASP top 10

Fig. 1.3 – Logo et icônes de l'OWASP Top 10

1.3 Tests de sécurité

Les tests de sécurité jouent un rôle primordial dans la protection des applications web contre les cyberattaques. Ils permettent de détecter les vulnérabilités dès la phase de développement, ce qui facilite leur correction avant mise en production. Intégrés au cycle de vie du développement logiciel, ces tests réduisent les coûts des corrections tardives et assurent la conformité légale et réglementaire.

1.3.1 Méthodes de test

Les méthodes de tests se divisent en deux grandes catégories :

— Les tests manuels sont effectués par des testeurs humains, souvent des experts en assurance qualité ou en sécurité, selon le domaine. Ils consistent à explorer manuellement le logiciel, le système ou l'application afin de détecter des défauts ou des anomalies.

— Les tests automatisés sont réalisés à l'aide d'outils, de scripts ou de logiciels qui exécutent automatiquement des vérifications sur le système à tester. Ils sont particulièrement utiles pour les tâches répétitives ou à grande échelle.

1.3.2 Tests manuels vs automatisés

Le tableau 1.2 présente un tableau comparatif entre les deux types de test : manuel et automatisé. La comparaison est effectué dans le contexte de la sécurité des systèmes d'informations. La comparison est faite sous certain critères à savoir : le coût, la fréquence et auquel type d'attaques est adapté.

Tab. 1.2 – Tableau comparatif entre tests manuels et automatisés

Critère	Test Manuel	Test Automatisé
Définition	Réalisé par un expert humain	Exécuté par outils/scripts
Précision	Haute pour les vulnérabilités complexes	Bonne pour les failles connues
Temps d'exécution	Long	Rapide
Coût	Élevé (expertise humaine)	Plus faible une fois mis en place
Fréquence	Occasionnelle	Fréquente (CI/CD)
Reproductibilité	Faible	Élevée
Adapté pour	Attaques complexes	Détection continue

Les limitations majeures du test manuel sont :

— Temps et ressources élevés

— Erreurs humaines fréquentes

— Difficulté de répétabilité

1.4 Étude et critique de l'existant

Le tableau 1.3 présente une synthèse des principales catégories d'outils utilisés pour améliorer la sécurité des applications, avec des exemples concrets, une description fonctionnelle et les limites associées à chaque type d'outil.

Tab. 1.3 – Outils de sécurité existants

Type de logiciel	Exemples	Description	Limites
Analyse statique du code	DeepSource, SonarCloud, Coverity	Contrôle de code source, détection de vulnérabilités et mauvaises pratiques	Coûts élevés pour les grands projets
Analyse des dépendances (SCA)	Snyk, Black Duck	Détecte vulnérabilités connues dans bibliothèques open source	Limitations sur plans gratuits, dépendance aux bases de CVE
Analyse dynamique (DAST)	OWASP ZAP, Burp Suite	Teste des applis en exécution (XSS, injections, etc.)	Configuration complexe, faux positifs possibles
Scan de dépendances (via CVE)	Dependency-Check, npm audit	Analyse fichiers de config pour détecter vulnérabilités connues	Ne détecte pas les failles zero-day

1.5 Solution proposée

Face aux limites des outils existants, le projet **CheckSec** vise à proposer une solution automatisée et intégrée pour l'orchestration de tests de sécurité. CheckSec automatise l'exécution de plusieurs outils,

collecte et centralise les résultats, et les affiche via une interface claire. L'objectif est une intégration fluide dans les processus DevOps.

La figure 1.4 représente le logo de notre application CheckSec

Fig. 1.4 – Logo de CheckSec

1.6 Intégration de DevOps dans les tests de sécurité

1.6.1 Contexte DevOps

L'intégration de DevOps dans les tests de sécurité repose sur un ensemble de pratiques visant à rapprocher les équipes de développement (Dev) et d'exploitation (Ops), dans une logique de collaboration continue et d'automatisation. DevOps s'appuie sur plusieurs piliers techniques tels que l'intégration et le déploiement continu (CI/CD), l'Infrastructure as Code (IaC), les pipelines d'automatisation, l'architecture en microservices ainsi que la surveillance et l'observabilité des systèmes. Cette approche favorise une culture organisationnelle axée sur la coopération entre les équipes, renforcée par des rituels agiles, des objectifs communs et une communication fluide. L'automatisation joue un rôle central dans cette démarche, en permettant de sécuriser et d'accélérer les processus de test, de déploiement et de configuration. Elle contribue à limiter les erreurs humaines tout en garantissant une meilleure réactivité face aux vulnérabilités, intégrant ainsi la sécurité de manière proactive dans le cycle de développement logiciel.

1.6.2 Intégration continue (CI) et déploiement continu (CD)

L'intégration continue permet d'exécuter automatiquement les tests à chaque modification du code. Le déploiement continu pousse ensuite les changements validés en production. Dans le cadre du projet CheckSec, les outils comme Trivy, Snyk, OWASP Dependency-Check, ou SonarCloud sont intégrés directement dans le pipeline CI/CD.

1.6.3 Surveillance

DevOps met un accent particulier sur la surveillance continue des applications en production et la collecte de feedback pour améliorer les processus et les produits. Les outils de surveillance permettent de détecter rapidement les problèmes et d'y remédier avant qu'ils n'affectent les utilisateurs.

1.6.4 Cycle de vie DevOps

Le cycle de vie DevOps comprend les phases suivantes [1] :

— **Plan :** Il s'agit de la première étape de la pratique DevOps. Ici, l'équipe de développement définit les exigences et les objectifs du projet et établit un plan d'exécution.

— **Code :** L'équipe de développement travaille sur le code à l'aide d'outils de contrôle de version tels que Git, Mercurial et Subversion pour la collaboration.

— **Build :** Le code écrit est compilé et rendu exécutable pour le déploiement.

— **Test :** Des tests automatisés sont effectués sur le code pour vérifier les bugs et les erreurs, et également pour s'assurer que le code fonctionne comme prévu.

— **Déployer :** Le logiciel est déployé à l'aide du pipeline de livraison continue (CD) pour permettre la publication fréquente de fonctionnalités en production.

— **Exploiter :** Dans cette phase, l'équipe des opérations teste le logiciel dans un environnement de production pour valider qu'il convient aux utilisateurs finaux.

— **Observer et surveiller :** Les développeurs et les équipes opérationnelles reçoivent une surveillance et des retours continus, ce qui leur permet d'identifier et de résoudre rapidement les problèmes.

Les pratiques DevOps visent à améliorer la collaboration entre les équipes de développement et d'exploitation. Elles incluent :

– **Intégration continue :** Intégration fréquente des modifications de code avec des tests automatisés pour détecter et corriger les problèmes rapidement.

– **Livraison continue :** Automatisation du processus de livraison pour permettre le déploiement en production de chaque modification de code, avec une surveillance continue.

– **Déploiement continu :** Extension de la livraison continue où chaque modification est automatiquement déployée en production, réduisant le délai entre validation et mise en production.

Ces pratiques permettent d'accélérer le développement, d'améliorer la qualité du logiciel et de fournir des fonctionnalités plus rapidement et fiablement aux utilisateurs.

La figure 1.5 présente le cycle de vie de DEVOPS.

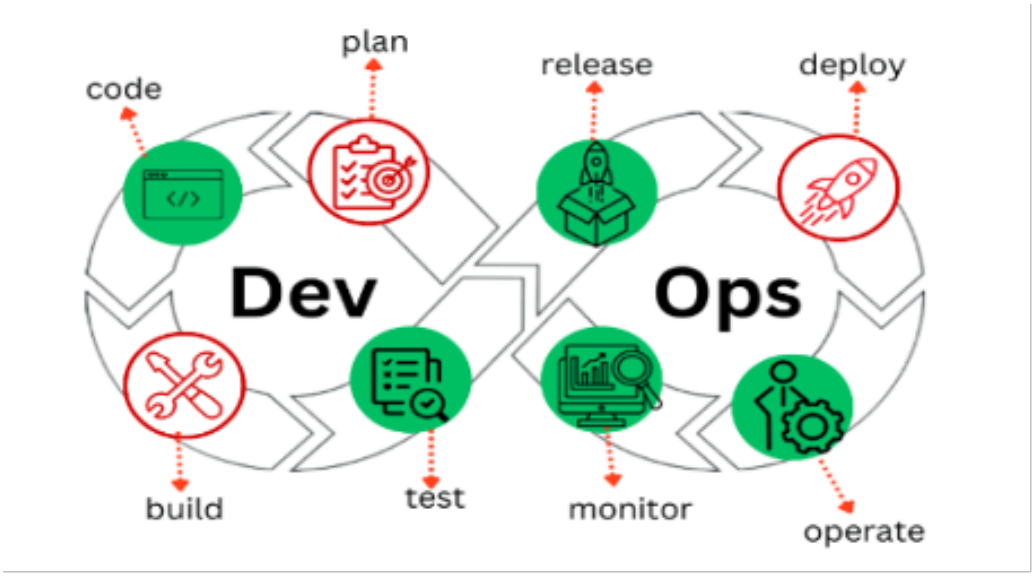

Fig. 1.5 – Cycle de vie de DEVOPS

1.7 Méthodologie adoptée

Dans tout projet informatique, il est essentiel d'adopter une démarche structurée pour assurer une gestion efficace et la création d'un produit fiable. Cette approche vise à répondre aux attentes des clients tout en détaillant le processus de développement de l'application.

1.7.1 Choix de la méthode

Au cœur de notre projet se trouve *CheckSec*, une solution dédiée à l'automatisation des tests de sécurité pour les applications web. Conscients de l'évolution rapide de ce domaine, nous avons choisi la méthodologie Agile Scrum, qui offre flexibilité et capacité d'adaptation aux changements. Cette méthode permet d'avancer efficacement tout en gardant pour objectif une solution sécurisée, automatisée et accessible.

1.7.2 Les méthodes agiles

Les méthodes agiles [3] raccourcissent les cycles de développement et permettent une réponse rapide aux demandes changeantes des utilisateurs. Elles sont incrémentales, itératives et adaptatives, ce qui les rend particulièrement efficaces dans les environnements dynamiques.

1.7.3 Présentation de la méthodologie Scrum

Scrum [4] repose sur une équipe soudée orientée vers un objectif commun, avec des itérations appelées *sprints* (2 à 4 semaines) aboutissant à un produit fonctionnel.

Ici, dans cette section La figure 1.6 montre les concepts fondamentaux de la méthodologie Scrum

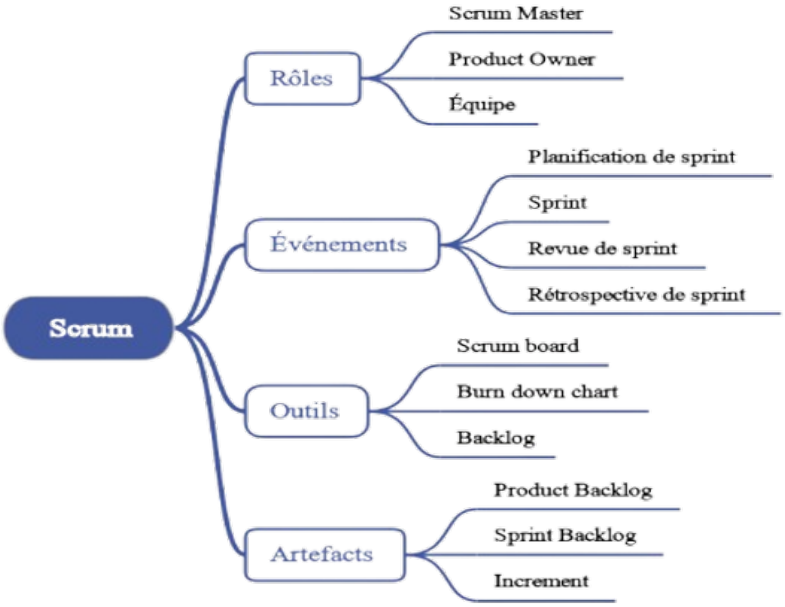

FIG. 1.6 – Les concepts fondamentaux de SCRUM

1.7.4 Les rôles de Scrum

Les trois rôles fondamentaux sont :

- **Product Owner :** Définit les objectifs métier et priorise le backlog produit.
- **Scrum Master :** Garant du respect des pratiques Scrum, résout les blocages et favorise l'amélioration continue.
- **Équipe de développement :** Multidisciplinaire, elle conçoit, développe, teste et livre les incréments de produit.

1.7.5 Les événements de Scrum

- **Planification du Sprint :** Définition des tâches pour le prochain sprint.
- **Sprint :** Période de travail de 2 à 4 semaines.
- **Revue du Sprint :** Présentation des résultats et collecte de feedback.

– **Rétrospective du Sprint :** Analyse de ce qui a fonctionné ou non pour s'améliorer continuellement.

1.7.6 Les outils de Scrum

– **Scrum board :** Tableau visuel pour suivre l'avancement des tâches.
– **Burn down chart :** Graphique représentant le travail restant sur la durée du sprint.
– **Backlog :** Liste évolutive des fonctionnalités, tâches et améliorations à effectuer.

1.7.7 Les artefacts de Scrum

– **Product Backlog :** Liste ordonnée des besoins du produit.
– **Sprint Backlog :** Liste des éléments à réaliser durant le sprint.
– **Incrément :** Fonctionnalités livrées et testées à la fin du sprint.

1.7.8 Fonctionnement de Scrum

Dans cette partie, nous présentons la figure 1.7 qui illustre le cycle de fonctionnement de la méthode Scrum, mettant en évidence les différentes étapes clés du processus ainsi que leur enchaînement logique.

Fig. 1.7 – Fonctionnement de SCRUM

Le fonctionnement Scrum repose sur les étapes suivantes :

– Le Product Owner alimente et priorise le Product Backlog.
– L'équipe Scrum sélectionne les éléments à développer dans le Sprint Backlog.
– Pendant le Sprint, les tâches sont réalisées, et un incrément de produit est produit.
– Une revue du Sprint permet de présenter les résultats et de collecter les feedbacks.
– Une rétrospective permet de tirer des leçons et d'améliorer les pratiques.
– Le backlog est raffiné en continu selon les retours et les priorités.

Conclusion

Ce premier chapitre a présenté le cadre général du projet en mettant en avant l'importance de la cybersécurité et les limites des tests de sécurité traditionnels. La solution proposée, *CheckSec*, vise à répondre à ces défis avec une approche automatisée et agile.

Dans le chapitre suivant, nous aborderons la phase initiale du projet, appelée *Sprint 0*, qui inclut la définition du plan de travail, la constitution du backlog et la sélection des outils. Cette étape est essentielle pour structurer efficacement la suite du projet.

CHAPITRE 2

SPRINT 0 : Etude techniques

Sommaire

Introduction 16

2.1 Spécification des besoins 16

2.1.1 Les besoins fonctionnels 16

2.1.2 Spécification semi-formelle des besoins 17

2.1.3 Besoins non fonctionnels 18

2.1.4 Environnement de développement et choix techniques 19

2.2 Architecture générale de l'application 23

2.3 Conception détaillée 25

2.3.1 Aperçu des classes du projet 25

2.3.2 Diagramme de séquence d'authentification 26

2.3.3 Diagramme de séquence d'inscription 27

2.4 Architecture CI/CD 27

2.4.1 Pipeline CI/CD 28

2.4.2 Mise en place de GitHub 28

Conclusion 29

Introduction

Dans ce chapitre, nous analysons les solutions existantes pour identifier des axes d'amélioration et présenter ces idées en tant que fonctionnalités. Nous déterminons les acteurs impliqués et leurs tâches respectives, tout en abordant les besoins fonctionnels et non fonctionnels liés à la première phase de Scrum. Cela inclut l'environnement de travail, les choix techniques et l'architecture du projet.

2.1 Spécification des besoins

Il est crucial de bien comprendre le contexte d'un projet avant de se lancer dans sa réalisation. Pour cela, une étape primordiale consiste à spécifier les besoins, à savoir les besoins fonctionnels et non fonctionnels de la plateforme, ainsi qu'à identifier les différents acteurs impliqués dans le projet. Cette étape permet d'avoir une vision claire et précise des attentes des utilisateurs finaux.

2.1.1 Les besoins fonctionnels

— **Le product backlog** :

Le Product Backlog regroupe l'ensemble des tâches, fonctionnalités et améliorations nécessaires pour atteindre l'objectif spécifique de notre projet. Il représente la référence unique pour l'équipe et assure que seules les tâches présentes sont exécutées.

— **ID :** représente le numéro d'identification de user story.

— **User Story :** décrit un comportement du produit visible pour les utilisateurs qui apporte une valeur ajoutée.

— **Priorité :** la priorité de user story qui est affectée en fonction de la valeur qu'elle apporte au produit et de l'ordre de réalisation. Le numéro 1 représente la priorité ayant le niveau le plus haut.

— **Complexité** : représente le degré de complexité d'une User Story. Nous utiliserons la suite de Fibonacci comme moyen d'évaluation (1, 2, 3, 5, 8, 13), pour la raison suivante : la différence entre la valeur 2 et 3 est plus significative (+50 %) que la différence entre la valeur 13 et 14 (+8 %) même si la différence absolue est la même (+1). Cela nous aide à évaluer la complexité plus facilement car, en général, les êtres humains ne sont pas très bons pour évaluer les valeurs absolues.

Tab. 2.1 – Backlog

ID	User Story	Priorité	Complexité
1	En tant qu'utilisateur, je désire la possibilité de créer un compte de manière sécurisée dans l'application en ajoutant : Prénom, Nom, Email, Mot de passe, Confirmation de mot de passe.	H	5
2	En tant qu'utilisateur, je désire la possibilité de m'authentifier de manière sécurisée dans l'application en utilisant mon adresse e-mail et mon mot de passe.	H	5
3	En tant qu'utilisateur, je désire la possibilité d'accéder à mon Dashboard après l'authentification.	H	3
4	En tant qu'utilisateur, je désire la possibilité de faire un scan en ajoutant le dépôt GitHub à scanner et accéder à la page "résultats de scan".	H	8
5	En tant qu'utilisateur, je souhaite accéder à une page "À propos" pour m'informer sur l'application, et à une page "Nous contacter" pour pouvoir contacter l'équipe en cas de problème ou de question.	B	2
6	En tant qu'utilisateur, je désire la possibilité d'automatiser les résultats de scan pour chaque changement de code.	H	8
7	En tant qu'utilisateur, je désire le déploiement automatique de mon application.	H	8

2.1.2 Spécification semi-formelle des besoins

Dans cette section, nous décrivons le rôle de l'acteur principal ainsi que sa déscription, comme illustré dans le tableau 2.2.

Tab. 2.2 – Résumé des rôles

Rôle	Description
Utilisateur	— Peut créer un compte — Peut accéder à son propre compte — Peut accéder à l'interface « À propos » — Peut accéder à l'interface « Nous contacter » — Peut faire un scan d'un dépôt GitHub

Le diagramme de cas d'utilisation de notre application Checksec donne un aperçu sur les parties qui seront developpées par la suite. La figure 2.1 détaille les différentes fonctionnalités offertes par Checksec. Un utilisateur peut afficher le résultat de son scan via différents outils :Snyk , Trivy , SonarCloud et OWASP dependency check.

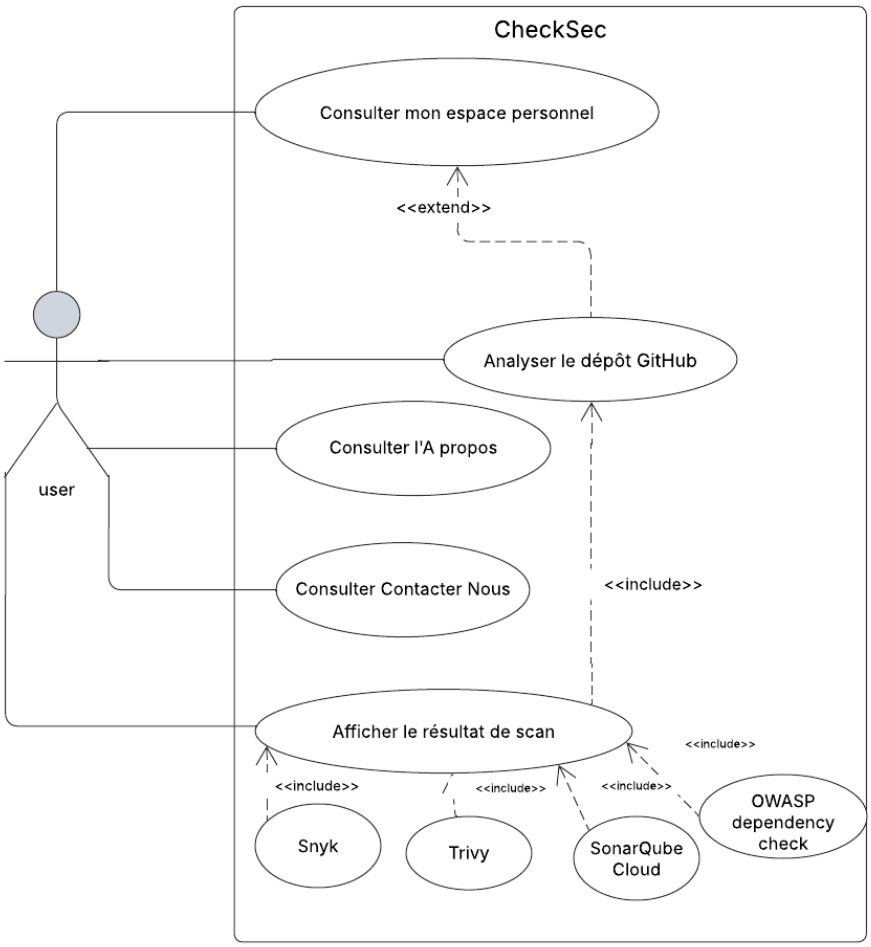

Fig. 2.1 – Diagramme de cas d'utilisation de CheckSec

2.1.3 Besoins non fonctionnels

Les exigences non fonctionnelles sont des spécifications qui n'expriment pas une fonctionnalité du système. Cependant, elles fournissent des contraintes qui doivent être respectées. En fait, notre futur système doit satisfaire les critères indiqués dans le tableau 2.3 ci-dessous.

Tab. 2.3 – Besoins non fonctionnels

Besoins non fonctionnels	**Description**
Sécurité	Seuls les utilisateurs autorisés doivent pouvoir accéder aux fonctionnalités critiques du système.
Accessibilité	L'interface utilisateur doit être simple et accessible même à des utilisateurs non techniques. Les résultats doivent être lisibles et compréhensibles, avec une légende si besoin.
Portabilité / Déploiement	Le système doit pouvoir être déployé sur différentes plateformes (Docker, Render, etc.). Le déploiement doit être automatisé à l'aide de GitHub Actions.

2.1.4 Environnement de développement et choix techniques

Dans cette partie, nous présenterons l'environnement matériel utilisé pour développer ce projet, ainsi que l'environnement logiciel.

Environnement matériel

Au cours du développement de ce projet, nous avons utilisé un ordinateur Lenovo avec les spécifications présentées dans le tableau 2.4.

Composant	Spécification
Ordinateur portable	Lenovo ThinkPad E16 Gen 1
CPU	Intel® Core™ i7-1355U (13e génération)
RAM	16,0 Gio (16 Go)
Disque dur	512,1 Go
GPU	Intel® Graphics (RPL-U)
Système d'exploitation	Ubuntu 24.04.2 LTS

Tab. 2.4 – Spécifications de l'ordinateur

Environnement de développement

Cette section présente les outils et les langages de programmation utilisés pour le développement de notre plateforme.

Outils de Frontend

Dans cette section , nous présentons les technologies utilisés pour le développement de frontend de Checksec. La figure 2.2 détaille les outils utilisés pour développer l'interface Checksec.

Fig. 2.2 – Les outils de Frontend

L'application utilise plusieurs technologies clés pour son développement :

— **Angular** : framework web structuré, modulaire, avec support de la programmation réactive, du routing et du data-binding.

— **JavaScript** : langage de programmation côté client pour l'interactivité dynamique des pages web.

— **HTML** : langage de balisage utilisé pour structurer les pages web.

— **CSS** : langage de style utilisé pour définir l'apparence visuelle et l'ergonomie.

Outils de Backend

Nous présentons ici les technologies utilisées pour le développement du backend de notre application. La figure 2.3 illustre les outils employés dans la partie backend de CheckSec.

Fig. 2.3 – Les outils de Backend

— **Express.js** : framework web léger basé sur Node.js pour créer des API RESTful.

— **Node.js** : environnement d'exécution JavaScript côté serveur, permettant la création d'applications rapides et évolutives.

Outils de base de données

Nous détaillons ici les technologies adoptées pour la gestion de la base de données de l'application. La figure 2.4 présente les différents outils utilisés dans l'architecture de la base de données de CheckSec.

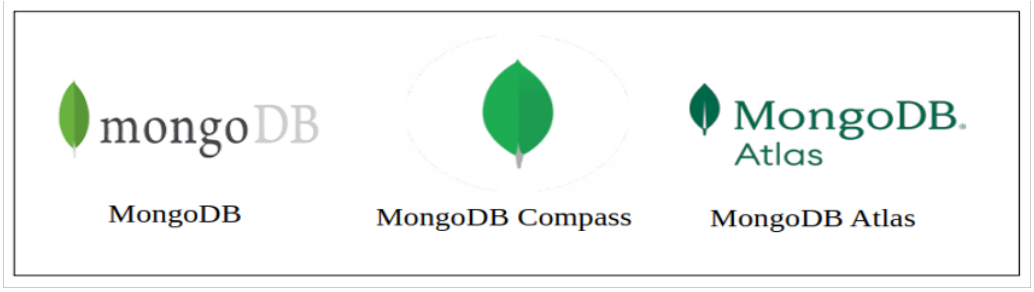

Fig. 2.4 – Les outils de Base de données

— **MongoDB** : système NoSQL orienté documents.

— **MongoDB Compass** : interface graphique officielle pour la visualisation des collections.

— **MongoDB Atlas** : plateforme cloud de déploiement et gestion des bases MongoDB.

Environnement logiciel

Nous présentons ici les logiciels qui ont été utilisés tout au long du développement de l'application. La figure 2.5 montre les principales technologies mises en œuvre dans les différentes étapes du projet CheckSec.

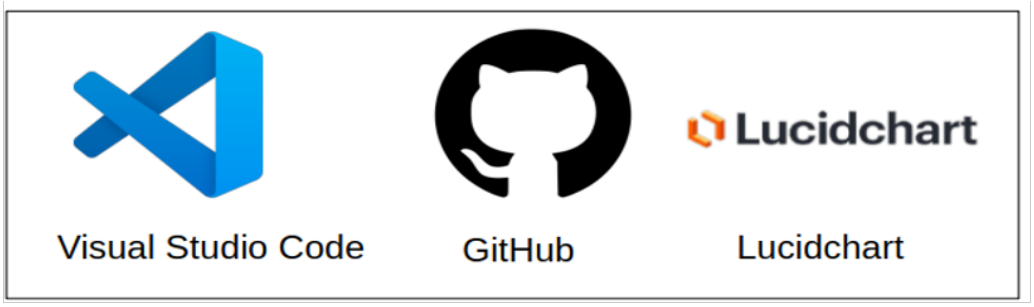

Fig. 2.5 – Les logiciels utilisés

— **VS Code** : éditeur de code léger, extensible, adapté au développement web et cloud.

— **GitHub** : plateforme de gestion de versions et collaboration (Git).

— **Lucidchart** : outil de création de diagrammes collaboratifs en ligne.

Outils de test

Nous présentons ici l'outil utilisé pour tester l'application. La figure 2.6 illustre cet outil, qui nous a permis de vérifier la bonne communication entre le frontend et le backend.

Fig. 2.6 – Postman

Postman est un outil de test et de documentation des API utilisé dans le cadre du projet pour simuler les requêtes, vérifier les réponses et automatiser certains tests fonctionnels.

Outils de scan de sécurité

Cette section décrit les outils utilisés pour réaliser les analyses de sécurité. La figure 2.7 illustre ces outils, qui ont permis de détecter et d'évaluer les vulnérabilités de l'application.

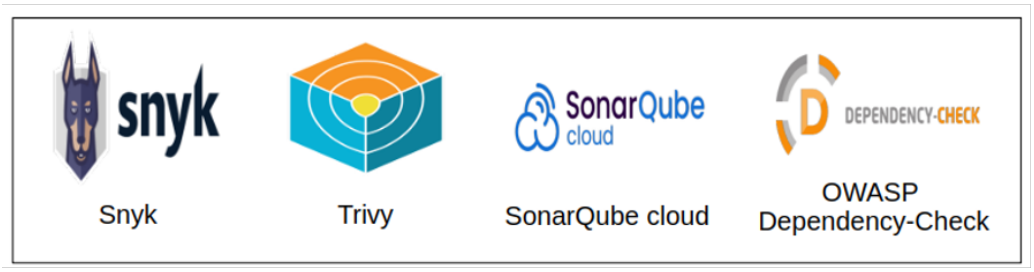

Fig. 2.7 – Les outils de scan de sécurité

- **Snyk** : plateforme de détection et correction des vulnérabilités dans les dépendances.
- **Trivy** : scanner de sécurité complet pour images Docker, dépendances et infrastructures.
- **SonarQube Cloud** : analyseur statique SaaS pour la qualité du code.
- **OWASP Dependency-Check** : outil de détection des vulnérabilités connues dans les dépendances.

Outils DevOps / Déploiement

Dans cette partie, nous expliquons les outils mis en œuvre pour les pratiques DevOps et le déploiement de l'application. La figure 2.8 présente ces technologies qui facilitent l'intégration, l'automatisation et la mise en production de CheckSec.

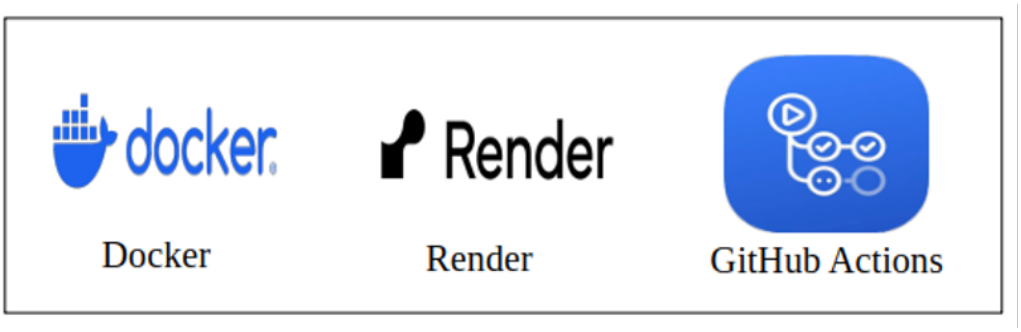

Fig. 2.8 – Les outils de DevOps

- **Docker** : conteneurisation pour une portabilité et une isolation des environnements.
- **Render** : plateforme PaaS pour le déploiement simplifié des applications.
- **GitHub Actions** : service CI/CD intégré à GitHub, permettant d'automatiser les tests, scans et déploiements.

2.2 Architecture générale de l'application

Après avoir établi l'environnement logiciel et sélectionné les différentes technologies de programmation utilisées, il est désormais essentiel d'obtenir une vision globale de l'architecture de l'application pour assurer l'alignement avec nos objectifs stratégiques. La figure 2.9 suivante présente une architecture globale de l'application.

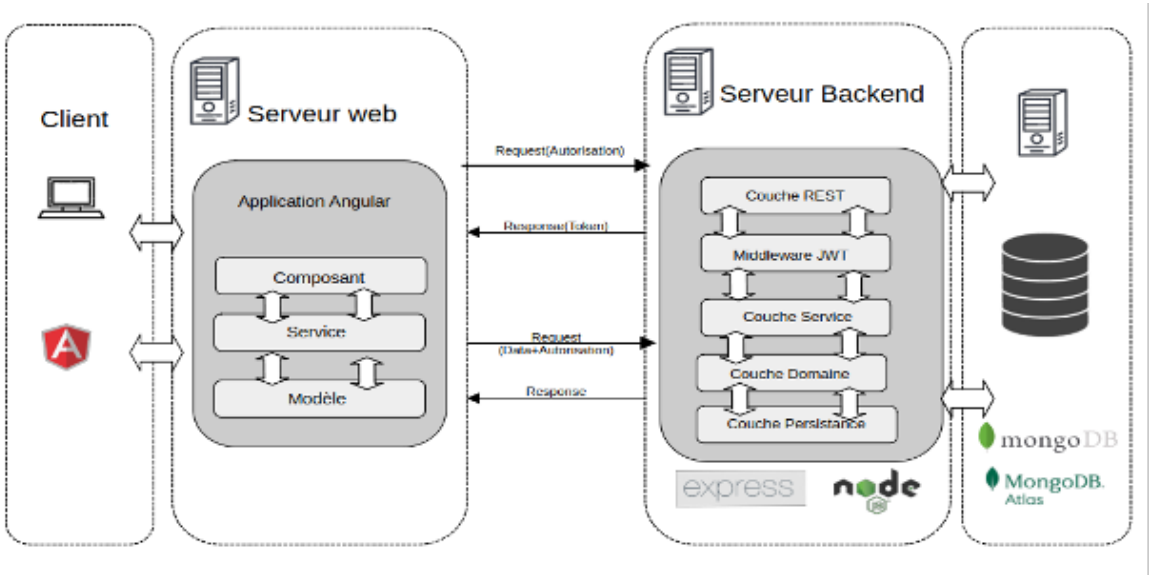

FIG. 2.9 – Architecture générale de l'application

L'application adopte une architecture client-serveur avec une séparation entre le frontend et le backend.

Frontend développé en Angular :

Développé avec Angular CLI, il est structuré en plusieurs éléments :

— **Composants** : représentent la vue et la logique de présentation ;

— **Services** : gèrent la logique métier, les appels API et le partage de données entre les composants ;

— **Modèles** : représentent les structures de données (interfaces ou classes).

TAB. 2.5 – Architecture Frontend

Élément	Rôle
Composants	Vue et logique de présentation
Services	Logique métier, appels API, partage de données
Modèles	Structure des données

Backend en Node.js avec Express :

Il adopte une architecture en couches (REST, middleware, service, domaine, persistance) pour une meilleure organisation, maintenabilité et évolutivité.

- **Couche REST** : gère les différentes routes (`/login`, `/sign-up`, `/scan-results`). Elle valide les données reçues, puis oriente la requête vers la logique métier appropriée.
- **Middleware JWT** : intercepte les requêtes pour appliquer des traitements transversaux, notamment la vérification des jetons d'authentification (ex. fichier `../middleware/authenticateToken`).
- **Couche Service** : contient la logique métier (vérification des identifiants, génération de token, traitement des scans) et orchestre les opérations complexes.
- **Couche Domaine** : définit les règles métier, objets métier (Utilisateur, Scan, etc.), et centralise la logique métier.
- **Persistance** : gère l'accès à la base de données MongoDB via Mongoose et réalise les opérations CRUD.

Tab. 2.6 – Architecture Backend

Couche	**Description**
REST	Gère les routes, validation, orientation vers services
Middleware JWT	Sécurise les routes privées par vérification des jetons
Service	Logique métier, orchestration des appels et règles globales
Domaine	Définition des entités métier et de leur logique propre
Persistance	Accès à MongoDB avec Mongoose, opérations CRUD

Base de données :

MongoDB (hébergée sur MongoDB Atlas), est une base NoSQL orientée documents. Elle est utilisée pour sa flexibilité. Mongoose est utilisé pour interagir avec la base, valider les données et gérer la sécurité (hachage des mots de passe, etc.).

2.3 Conception détaillée

2.3.1 Aperçu des classes du projet

Cette section présente les classes de l'application CheckSec. La figure 2.10 illustre les différentes classes.

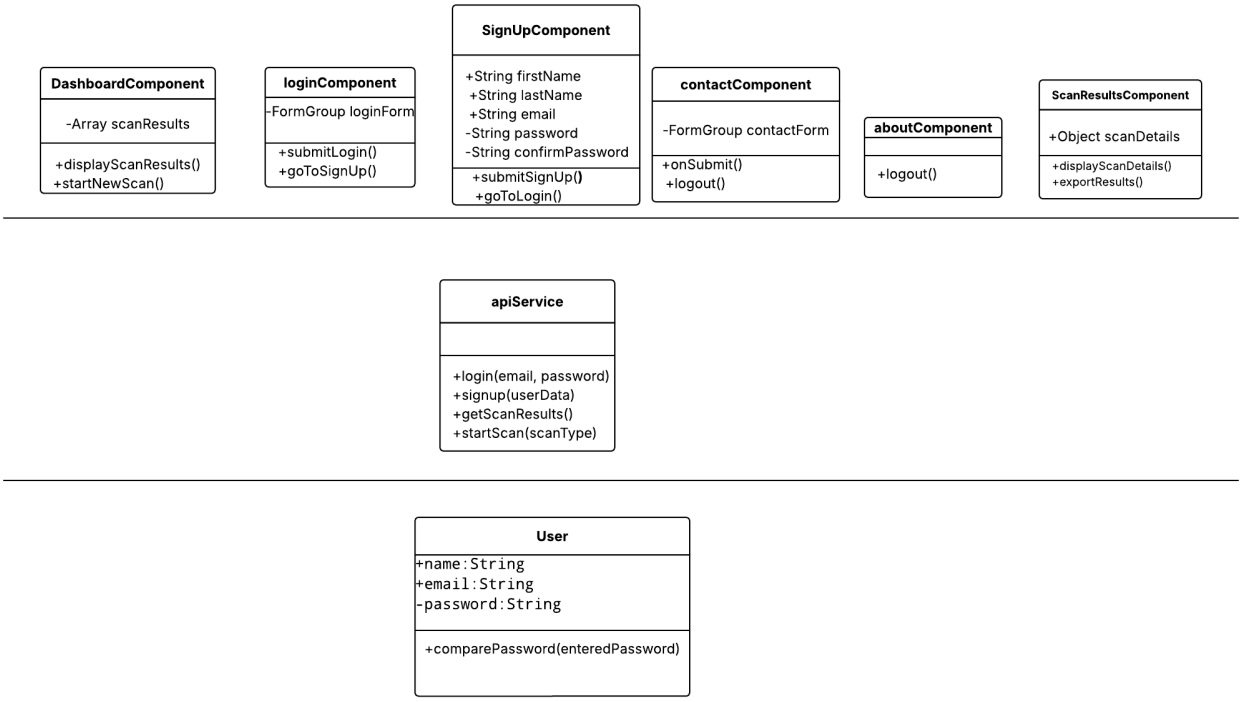

Fig. 2.10 – Les classes de CheckSec

2.3.2 Diagramme de séquence d'authentification

Cette section présente le diagramme de séquence du processus d'authentification de l'application CheckSec. La figure 2.11 montre les étapes permettant à un utilisateur de se connecter.

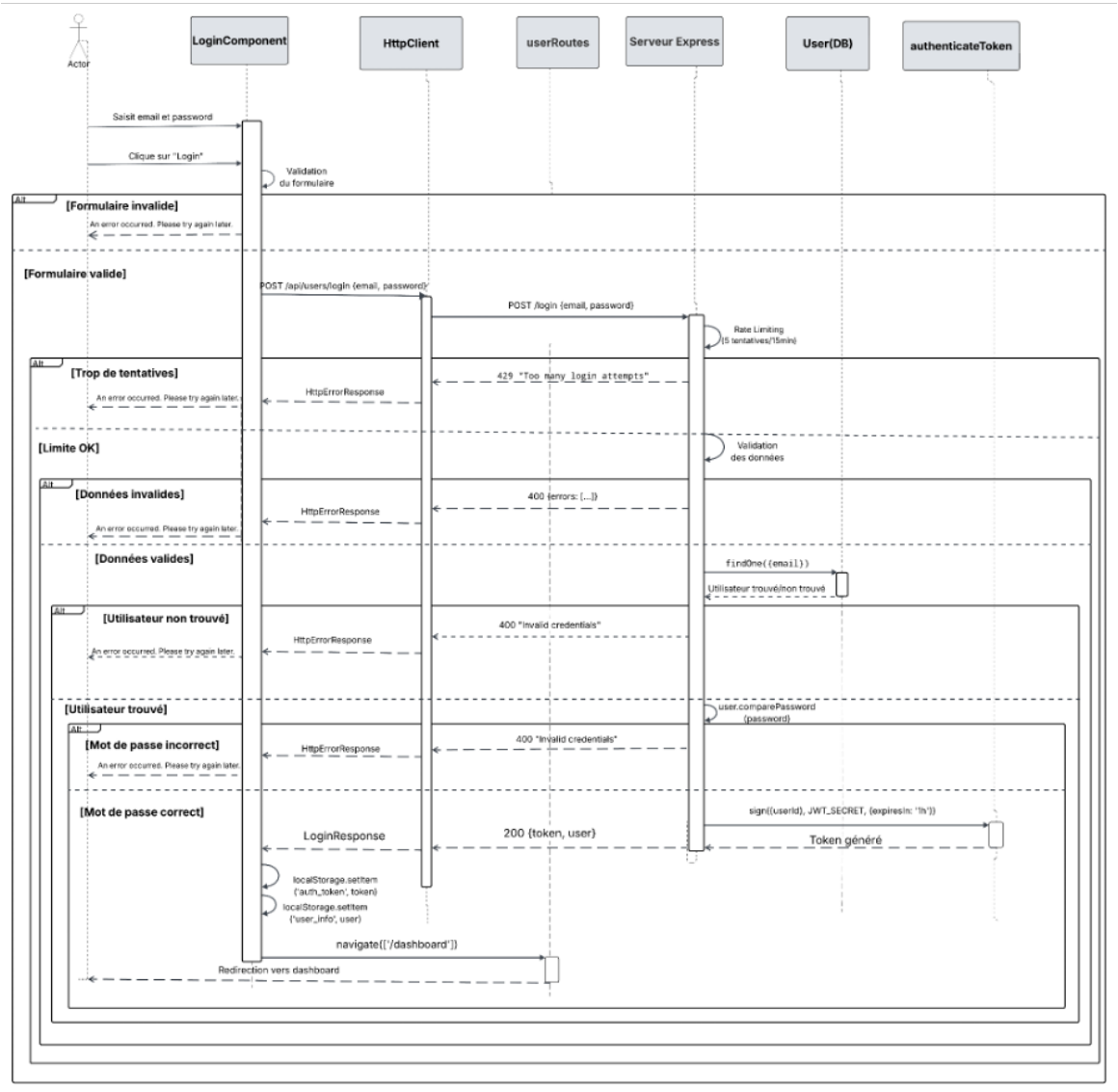

Fig. 2.11 – Diagramme de séquence d'authentification

2.3.3 Diagramme de séquence d'inscription

Cette section présente le diagramme de séquence du processus d'inscription. La figure 2.12 illustre comment un utilisateur peut créer un compte pour accéder à l'application.

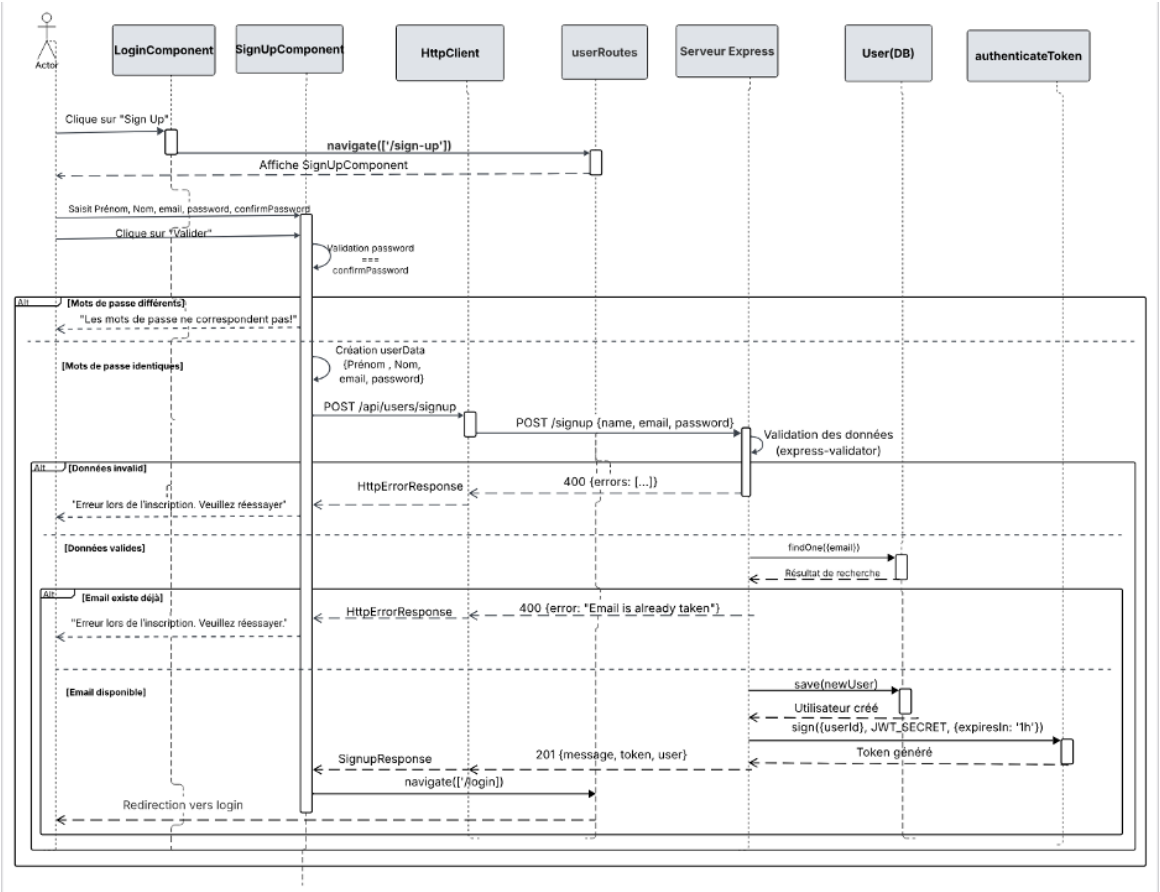

Fig. 2.12 – Diagramme de séquence d'inscription

2.4 Architecture CI/CD

Dans cette section, nous décrivons l'architecture CI/CD adoptée pour CheckSec. La figure 2.13 met en évidence les différentes étapes du processus, allant de la gestion du code source aux tests automatisés, jusqu'au déploiement final de l'application.

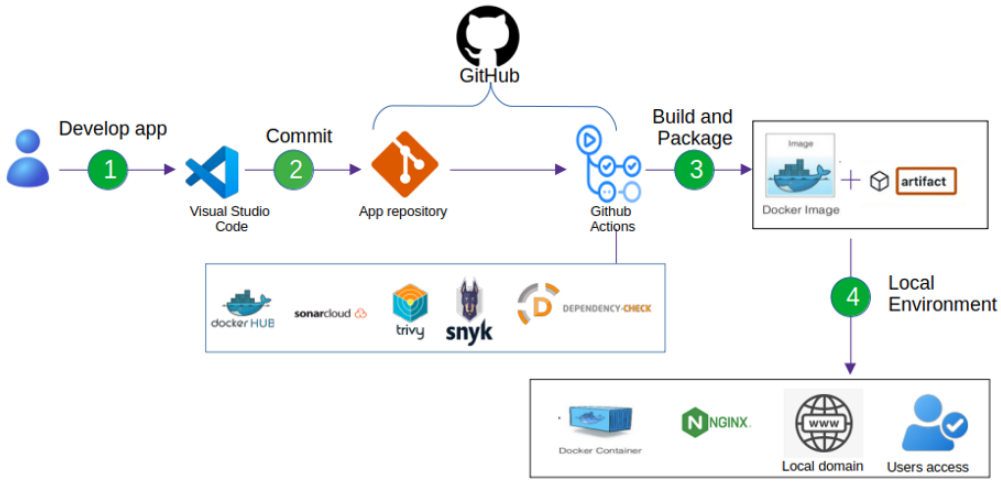

FIG. 2.13 – Architecture CI/CD

2.4.1 Pipeline CI/CD

La figure 2.13 montre le pipeline qui illustre un processus de développement et de déploiement automatisé (CI/CD) en 4 étapes principales :

1. **Développement de l'application** : Utilisation de Visual Studio Code comme environnement de développement intégré.
2. **Gestion de version et commit** : Le code est committé vers un dépôt Git hébergé sur GitHub.
3. **Intégration continue avec GitHub Actions** :
   - Analyse de sécurité : SonarCloud, Trivy, Snyk, Dependency Check ;
   - Construction : Image Docker et génération d'artefact ;
   - Publication : Image publiée sur Docker Hub.
4. **Déploiement local** :
   - Container Docker [2] ;
   - Nginx comme serveur web ;
   - Domaine local configuré ;
   - Interface utilisateur accessible.

2.4.2 Mise en place de GitHub

GitHub a été utilisé comme outil de gestion de version et de collaboration, permettant de suivre l'évolution du code :

- **Lien du dépôt** : https://github.com/EyaJrah/pfe

- **Branches** :
  - `main` : branche stable ;
  - `checkmain` : branche de développement.
- **GitHub Actions** :
  - `.github/workflows/build.yml` : scan SonarCloud ;
  - `.github/workflows/owasp-check.yml` : scan OWASP Dependency Check ;
  - `.github/workflows/snyk-security.yml` : scan Snyk ;
  - `.github/workflows/trivy-scan.yml` : scan Trivy ;
  - `.github/workflows/docker-build-push.yml` : déploiement Docker.

Voilà ,l'interface du dépôt GitHub de CheckSec. La figure 2.14 illustre la structure de code, ainsi que les éléments liés à l'automatisation, comme les workflows et les fichiers de configuration.

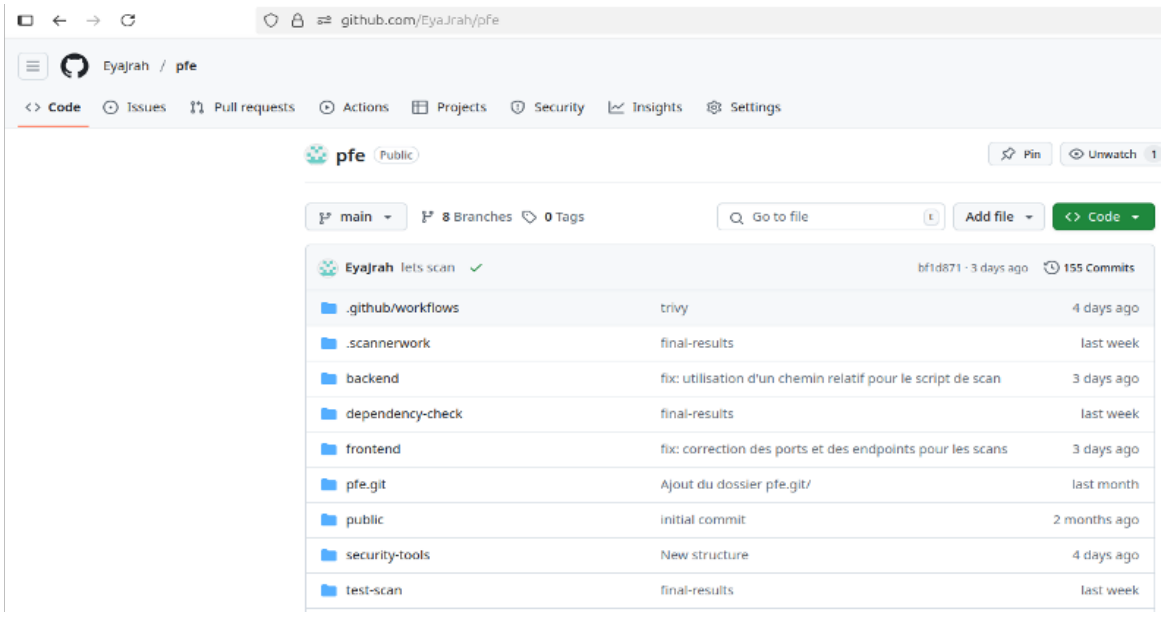

FIG. 2.14 – Interface du dépôt GitHub

Conclusion

Ce chapitre a permis de passer en revue les différentes étapes nécessaires à la spécification des besoins, à la conception de l'architecture, ainsi qu'aux choix techniques. Nous avons également présenté l'architecture physique de l'application `CheckSec`.

CHAPITRE 3

Sprint 1 : CI

Sommaire

Introduction 31

3.1 L'integration Continue dans CheckSec 31

3.2 Backlog du Sprint 1 31

3.2.1 User Story 01 : Inscription de l'utilisateur 32

3.2.2 User Story 02 : Authentification d'utilisateur 34

3.2.3 User Story 03 : Accès au Dashboard 36

3.2.4 User Story 04 : Accès au Résultat de scan 38

3.2.5 User Story 05 : Accès aux pages informatives 39

3.2.6 User Story 06 : Intégration et automatisation des tests de sécurité 41

3.3 Docker Build and Push : 47

Conclusion 47

Introduction

Au cours du chapitre précédent, nous avons présenté la planification de l'application. Ce chapitre est dédié au premier sprint, qui est entièrement consacré au développement, à l'analyse et à la mise en œuvre de l'intégration continue (CI), un élément essentiel de la chaîne CI/CD.

3.1 L'integration Continue dans CheckSec

La figure 3.1 montre les phases principales de l'intégration continue (CI).

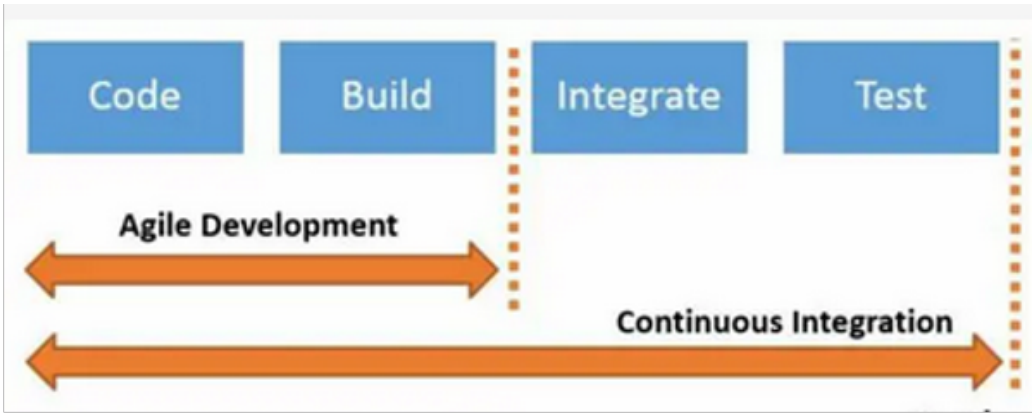

Fig. 3.1 – Les phases de CI

(https://training.genexus.com/en/learning/pdf/introduction-to-devops-with-genexus-pdf) L'intégration continue (CI) est une pratique de développement logiciel visant à automatiser les processus de compilation, de test et de vérification du code à chaque modification du dépôt.

Dans notre projet, l'intégration continue déclenche automatiquement une série de tests de sécurité, de qualité et fonctionnels, permettant de détecter rapidement toute vulnérabilité ou erreur dès l'introduction du code. Elle est mise en œuvre à l'aide de plusieurs flux GitHub Actions, configurés pour s'exécuter automatiquement sur la branche principale `main`. Ces flux intègrent des tests unitaires et des analyses de sécurité via des outils complémentaires, assurant ainsi la qualité et la fiabilité du code source.

3.2 Backlog du Sprint 1

Le backlog du premier sprint contient un ensemble de six User Stories priorisées selon leur importance pour l'utilisateur et le projet. Le tableau 3.1 détaille chaque User Story. Elle est identifiée par un identifiant unique (ID), décrite en termes de besoin utilisateur, et accompagnée d'une évaluation de priorité (Haute ou Basse).

ID	User Story	Priorité	Complexité
01	En tant qu'utilisateur, je désire pouvoir créer un compte de manière sécurisée en saisissant : prénom, nom, email, mot de passe, confirmation du mot de passe.	H	5
02	En tant qu'utilisateur, je désire pouvoir m'authentifier de manière sécurisée avec mon email et mon mot de passe.	H	5
03	En tant qu'utilisateur, je désire accéder à mon Dashboard après authentification.	H	3
04	En tant qu'utilisateur, je désire pouvoir effectuer un scan en ajoutant un dépôt GitHub, et accéder à une page "Résultats de scan".	H	8
05	En tant qu'utilisateur, je souhaite accéder à une page "À propos" et une page "Nous contacter".	B	2
06	En tant qu'utilisateur, je désire que les résultats de scan soient automatisés à chaque changement de code.	H	8

Tab. 3.1 – Backlog du sprint 1

3.2.1 User Story 01 : Inscription de l'utilisateur

L'objectif de cette User Story est de permettre à un nouvel utilisateur de créer un compte de manière sécurisée en saisissant les informations suivantes : prénom, nom, adresse email, mot de passe et confirmation du mot de passe. La figure 3.2 montre formulaire contient tous les champs requis.

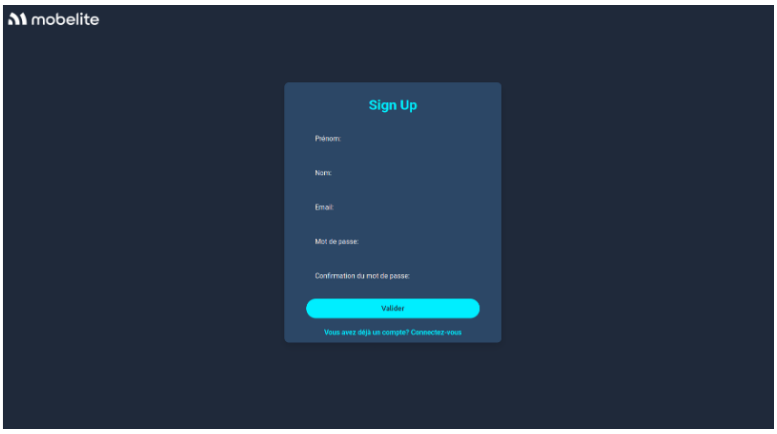

Fig. 3.2 – Interface d'inscription

Des validations sont effectuées côté client :

— Tous les champs sont obligatoires.

— L'email doit être au bon format.

— Le mot de passe doit comporter au moins 6 caractères.

— La confirmation du mot de passe doit correspondre au mot de passe saisi.

En cas d'erreur, un message explicite s'affiche sous le champ concerné.

Lors de la création de compte utilisateur, plusieurs contrôles sont effectués pour garantir la sécurité et la validité des données saisies. Les figures suivantes illustrent différents cas d'erreur rencontrés par

l'utilisateur :

— La Figure 3.3 montre une situation où un ou plusieurs champs obligatoires ne sont pas remplis. Le système affiche alors un message d'erreur clair pour informer l'utilisateur des champs manquants.

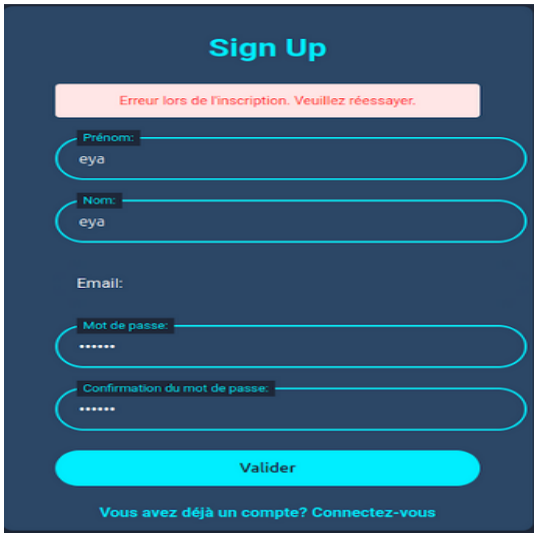

Fig. 3.3 – Champs manquant

— La Figure 3.4 présente un cas où l'utilisateur tente de créer un compte avec un email déjà utilisé dans la base de données. Le système empêche la duplication et notifie l'utilisateur que le compte existe déjà.

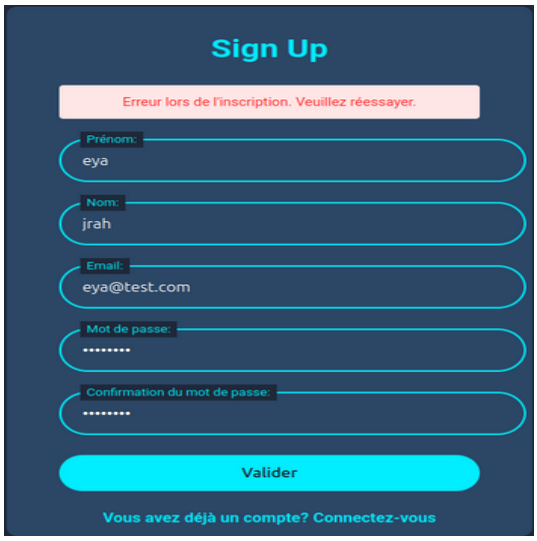

Fig. 3.4 – Utilisateur déjà existant

— La Figure 3.5 illustre une erreur de saisie lorsqu'un mot de passe de moins de six caractères est fourni. Un message d'erreur s'affiche afin d'imposer une longueur minimale, renforçant ainsi la sécurité des comptes.

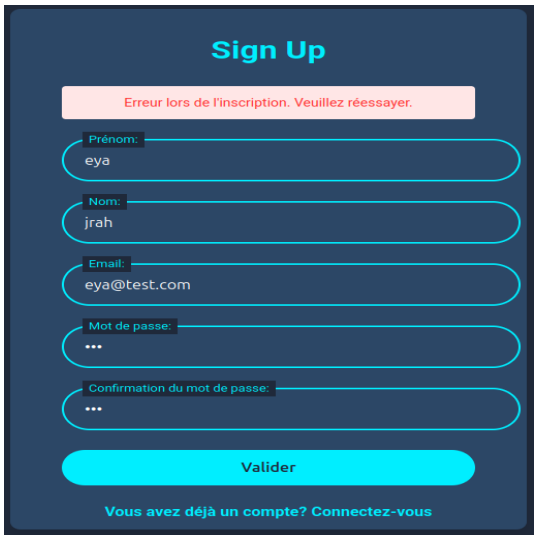

FIG. 3.5 – mot de passe à moins de six caractères

Pour la partie Backend, une route `POST /signup` a été ajoutée dans le backend. À la réception de la requête, le serveur exécute les vérifications suivantes :

— Vérifie que l'email n'est pas déjà utilisé (unicité).

— Vérifie que le mot de passe respecte la politique de sécurité.

— Vérifie que tous les champs sont présents et valides.

Si toutes les vérifications sont satisfaites :

— Le mot de passe est hashé avec `bcrypt` avant d'être stocké dans la base de données `MongoDB`. *(Le mot de passe n'est jamais stocké en clair.)*

— Un token `JWT` est généré et renvoyé au frontend pour connecter automatiquement l'utilisateur après son inscription.

3.2.2 User Story 02 : Authentification d'utilisateur

L'objectif est de permettre à un utilisateur existant de s'authentifier de manière sécurisée dans l'application en utilisant son adresse e-mail et son mot de passe.

La figure 3.6 illustre le formulaire de connexion, qui comporte deux champs : **email** et **mot de passe**.

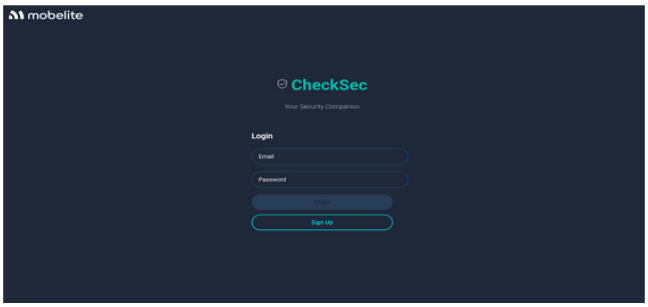

FIG. 3.6 – Interface d'Authentification

Le developpement de cette fonctionnalité du coté back comporte :

- Un composant de connexion a été créé sous le nom `login`.
- Des validations sont effectuées côté client : l'email doit être au bon format, le mot de passe est requis.
- Lors de la soumission, les données sont envoyées au backend via le service Angular `api.service.ts`.
- En cas d'erreur de saisie, un message explicite s'affiche à l'utilisateur :
  - Mot de passe ou Email incorrect
  - Email en mauvais format

La figure 3.7 illustre le message d'erreur affiché lorsque l'adresse e-mail saisie n'est pas dans un format valide.

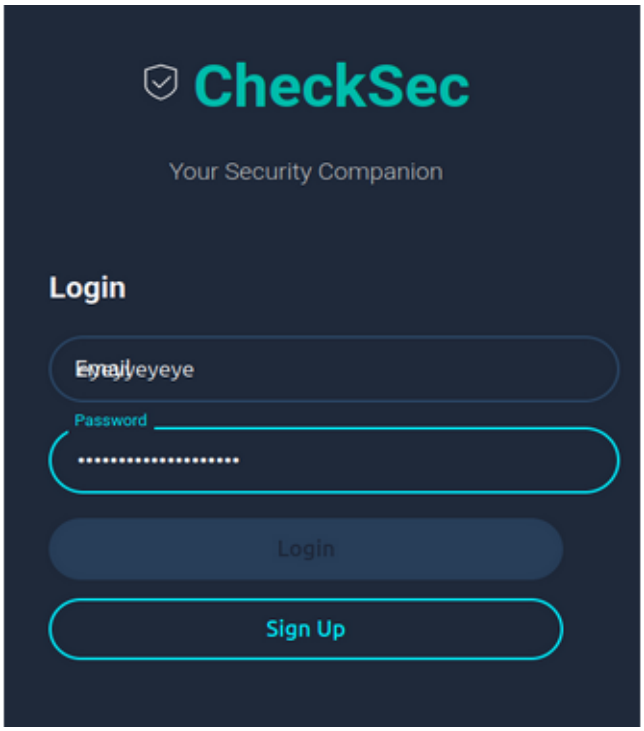

FIG. 3.7 – Interface d'Authentification avec erreur d'un email du mouvais format

La figure 3.8 présente le message d'erreur généré en cas de saisie incorrecte de l'adresse e-mail ou du mot de passe.

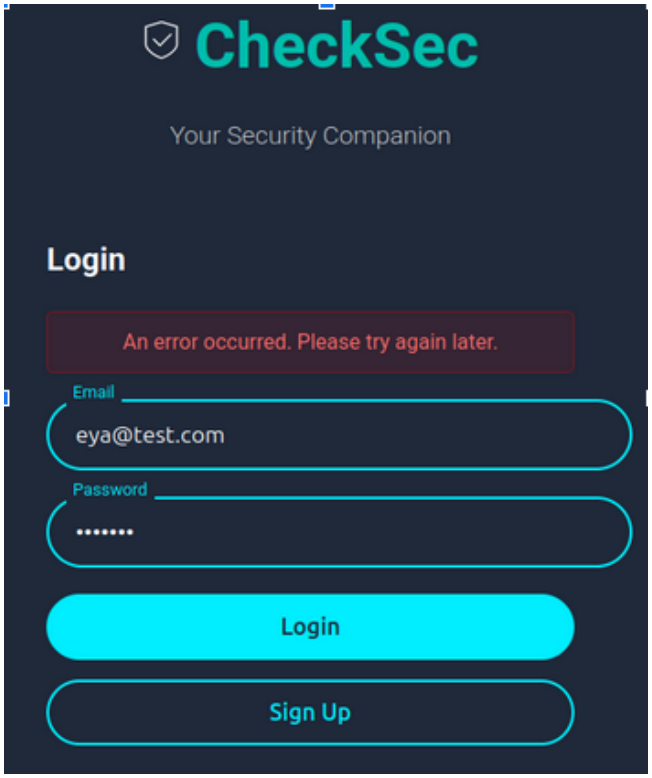

FIG. 3.8 – Interface d'Authentification avec erreur d'un email ou mot de passe incorrect

Le developpement de cette fonctionnalité du coté back comporte :

- Une route POST `/login` a été ajoutée.
- À la réception de la requête, le serveur :
  - Vérifie que l'email existe dans la base de données.
  - Compare le mot de passe fourni avec le hash stocké (grâce à la méthode `comparePassword` du modèle `User`).
  - Si les identifiants sont corrects, un token JWT est généré et renvoyé au frontend ; sinon, une erreur explicite est retournée.

3.2.3 User Story 03 : Accès au Dashboard

L'objectif de cette fonctionnalité est de permettre à l'utilisateur d'accéder à un espace personnel (dashboard) après authentification, où il peut visualiser ses informations et accéder aux principales fonctionnalités de l'application.

Le formulaire de dashboard représentée par la figure 3.9 comporte deux champs : un champ de saisie pour l'URL du dépôt GitHub et un bouton pour lancer le scan.

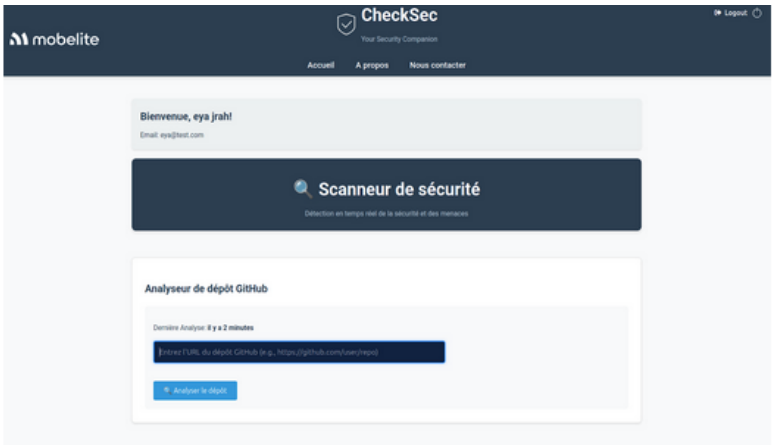

FIG. 3.9 – Interface de Dashboard

Développement côté Frontend (Angular) :

- Un composant `Dashboard` a été créé.
- Après connexion, l'utilisateur est redirigé vers le dashboard, qui affiche :
  - Un message de bienvenue personnalisé (ex. : "Bienvenue, [Nom] !").
  - L'adresse e-mail de l'utilisateur récupérée via le JWT stocké dans le `localStorage` ou depuis le backend.
- Des liens de navigation vers les principales fonctionnalités sont proposés.
- Dans la section scan :
  - L'utilisateur saisit l'URL du dépôt.
  - Un bouton "Lancer le scan" appelle la fonction Angular `startScan()`.
  - L'application vérifie que l'URL est valide, puis utilise `api.service.ts` pour lancer le scan.
  - Un indicateur de chargement est affiché pendant l'analyse.
  - En cas d'URL invalide, un message d'erreur s'affiche(voir figure 3.10).

FIG. 3.10 – En cas d'erreur d'URL

L'utilisateur sera par la ensuite redirigé vers le composant `Scan-Result`. Pour ce faire :

- Une route POST `/api/scan-results/scan-all` a été ajoutée (fichier `routes/scanResults.js`), protégée par `authenticateToken`.

— Le backend :
  — Reçoit l'URL du dépôt depuis le corps de la requête.
  — Vérifie le script `scan-and-send.sh`, le rend exécutable si besoin, puis l'exécute avec l'URL en paramètre.
  — Lance les outils Trivy, Snyk, SonarCloud, OWASP Dependency-Check.
  — Lit les résultats, les parse en JSON, les envoie au frontend.
— En cas d'erreur (URL manquante, script échoué, parsing incorrect), un message explicite est renvoyé.

3.2.4 User Story 04 : Accès au Résultat de scan

La figure 3.11 présente à l'utilisateur les résultats détaillés du scan pour le dépôt GitHub sélectionné.

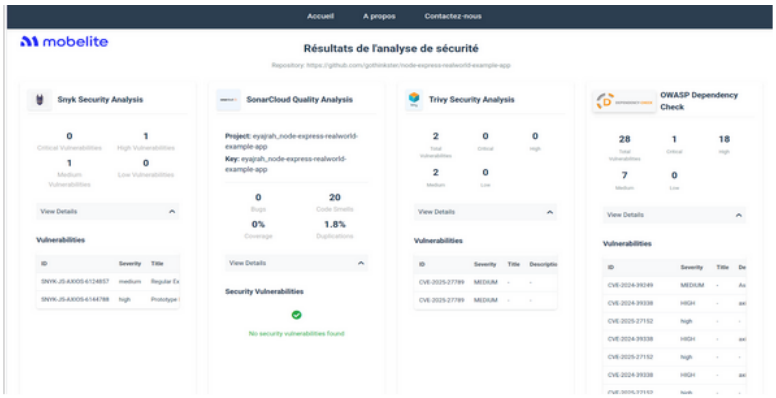

FIG. 3.11 – Interface de résultat de scan

La partie front developpée en Angular comporte :

— Le composant `scan-results` récupère l'URL et appelle `scan.service.ts` (GET /api/scan-results/scan-all).
— Les résultats sont stockés et affichés sous forme de tableau ou liste détaillée (Trivy, Snyk, etc.).
— En cas d'erreur ou absence de résultats, un message s'affiche.
— Un bouton permet de revenir au dashboard ou de lancer un nouveau scan.

Voilà ,la figure 3.12 montre le message affiché en cas d'erreur lors du scan.

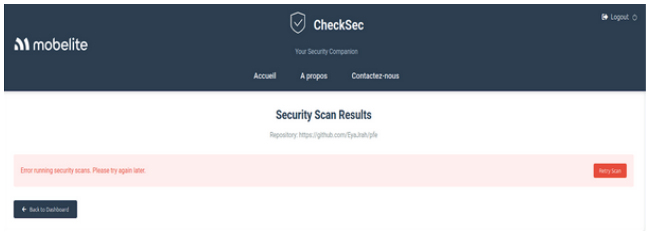

Fig. 3.12 – En cas d'erreur de scan

La partie Back developpée en Node.js/Express comporte :

- Le backend vérifie et exécute `scan-and-send.sh` avec l'URL en argument.
- Le script :
  - Clone le dépôt, lance les outils de sécurité, agrège les résultats dans un fichier log.
  - Affiche le chemin du fichier dans la sortie standard.
- Le backend lit, parse et renvoie les données JSON au frontend, puis supprime le fichier log.
- En cas d'erreur, un message explicite est envoyé.

3.2.5 User Story 05 : Accès aux pages informatives

Cette fonctionnalité offre à l'utilisateur des pages informatives et un moyen de contact simple avec l'équipe (voir les figures 3.13 et 3.14). La page À propos présente une vue d'ensemble du projet, son objectif principal ainsi que le contexte dans lequel il a été développé. La figure 3.13 montre la page « À propos » de CheckSec affichée à l'utilisateur.

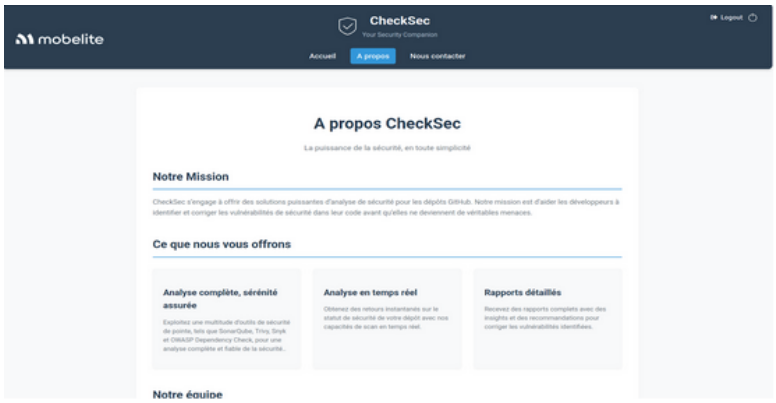

Fig. 3.13 – Interface de page À propos

La page Nous contacter met à disposition un formulaire simple permettant aux utilisateurs de poser des questions, signaler un problème ou donner un retour sur l'application.

La figure 3.14 montre la page « Nous Contacter » de CheckSec affichée à l'utilisateur.

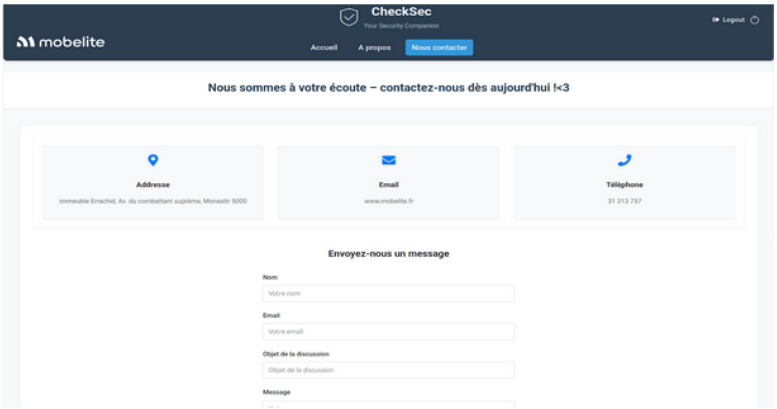

Fig. 3.14 – Interface de page Nous contacter

Pour pouvoir développer les deux dernières fonctionnalités, nous avons procédé comme suit :

— Deux composants : `about` et `contact` :

— `about.component.ts` présente l'application, les objectifs, les technologies et l'équipe.

— `contact.component.ts` propose un formulaire avec : nom, email, objet, message.

— Une validation s'assure que tous les champs sont remplis et corrects.

La figure 3.15 présente l'interface d'envoi du message avec succès.

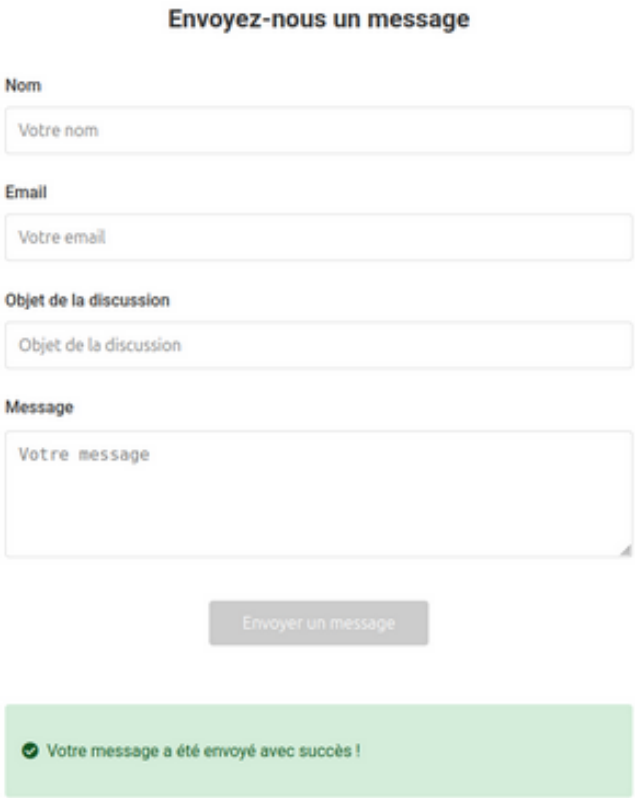

Fig. 3.15 – Message envoyé avec succès

La figure 3.16 confirme l'échec de l'envoi du message.

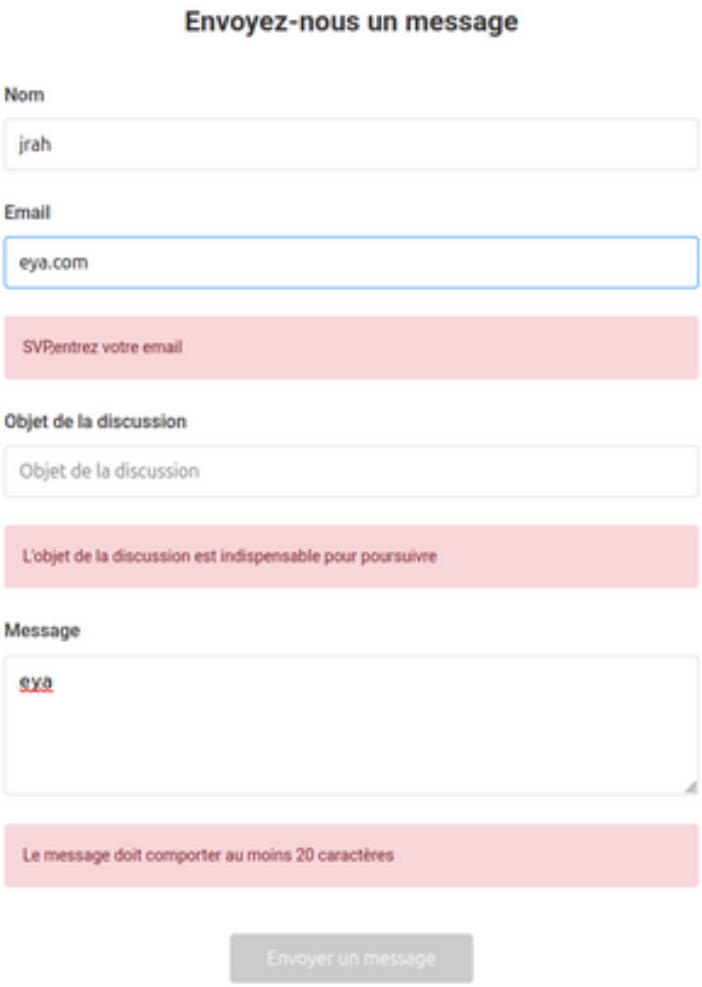

FIG. 3.16 – Erreur lors de l'envoi du message

Pour la fonctionnalité du contact, il s'agit de :

- Route POST `/api/contact` pour recevoir les données.
- Le backend valide les champs (présence, email valide, message non vide).
- Deux traitements possibles :
  - Envoi d'un email via `Nodemailer`.
  - Stockage en base de données.
- Réponse de succès ou message d'erreur selon le cas.

3.2.6 User Story 06 : Intégration et automatisation des tests de sécurité

Nous avons choisi GitHub Actions comme solution d'intégration continue, en raison de sa compatibilité directe avec GitHub, de sa flexibilité et de sa facilité de configuration. Le flux de travail que nous avons mis en place présente plusieurs avantages :

— Il est automatiquement déclenché par chaque requête *push* ou *pull*, garantissant une vérification continue du code.

— Il supporte plusieurs environnements (développement et production), facilitant la gestion des déploiements.

— Il intègre une gestion sécurisée des secrets (clés API, jetons d'accès) via GitHub Secrets, permettant une communication sécurisée avec des outils d'analyse tels que SonarCloud, Snyk, etc.

Cette figure 3.17 présente le logo de GitHub Actions

Fig. 3.17 – Logo de GitHub Actions

Le frontend, développé avec Angular, utilise Jest comme framework de test. La configuration est définie dans `jest.config.ts` et intégrée au pipeline CI/CD via GitHub Actions. Les tests sont exécutés automatiquement à chaque *push* ou *pull request* sur les branches `main`, `scanresult` et `checkmain`. Les tests actuels se concentrent sur les services essentiels, comme dans `navigation.service.test.ts`, où sont vérifiés :

— l'initialisation via `TestBed`,

— l'injection/configuration du `Router`,

— la création correcte du service,

— le bon fonctionnement des méthodes de navigation.

Cette phase de tests précède systématiquement l'exécution des workflows de sécurité (SonarCloud, Trivy, Snyk, OWASP). Elle inclut :

— la configuration de Node.js,

— l'installation des dépendances frontend et backend,

— l'exécution des tests unitaires.

Si les tests échouent, le pipeline s'arrête, empêchant toute analyse de sécurité sur un code instable.

Dans cette partie , la figure 3.18 présente les schémas des workflows pour chaque outil dans GitHub Actions

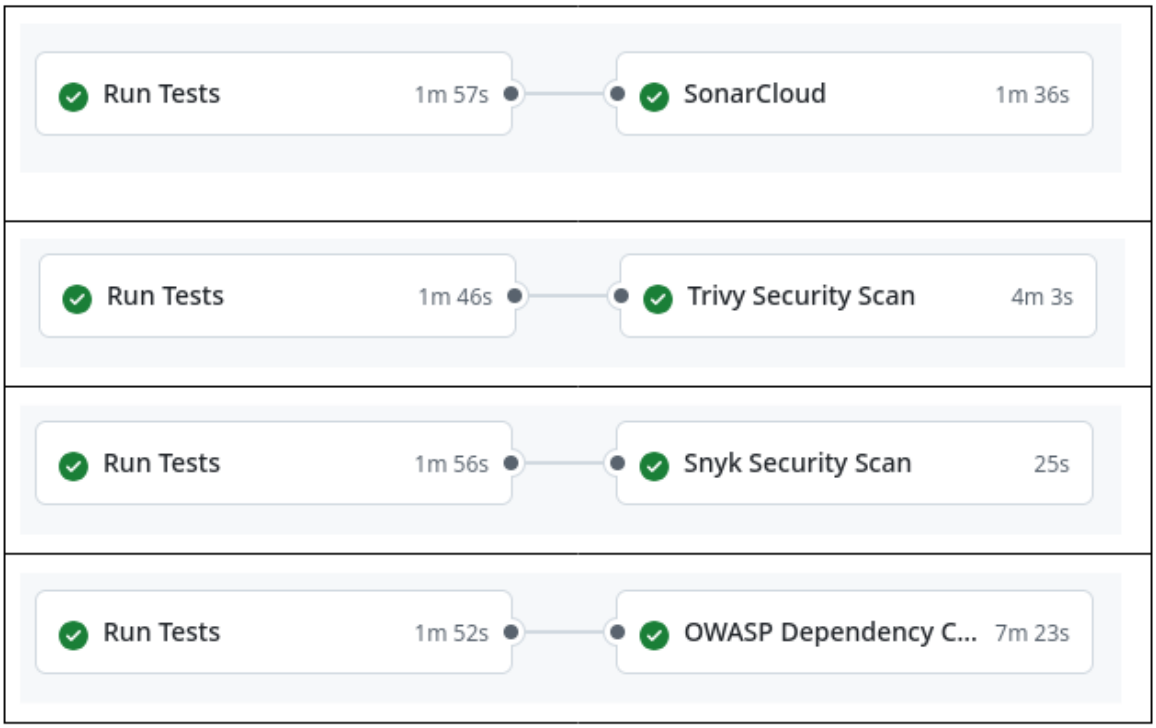

FIG. 3.18 – Schéma des tests unitaires et outils de sécurité dans GitHub Actions

SonarCloud :

SonarCloud assure une analyse continue de la qualité du code. Configuré dans `.github/workflows/build.yml`, il s'appuie sur :

— l'organisation : `eyajrah`,

— les projets : `eyajrah_backend`, `eyajrah_frontend`,

— un token stocké dans les *secrets* GitHub.

SonarCloud analyse automatiquement à chaque modification de code :

— les erreurs,

— les répétitions,

— les failles de sécurité,

— les parties non testées.

Les résultats sont disponibles sur SonarCloud, dans GitHub Actions et dans les *pull requests*.

La figure 3.19 représente le dashboard de SonarCloud

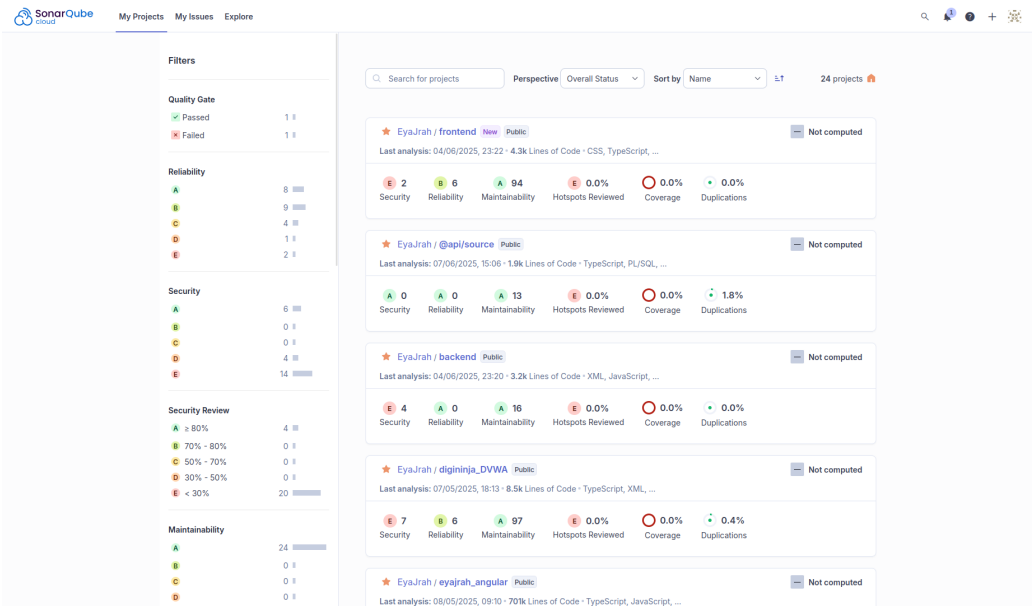

Fig. 3.19 – Tableau de bord de SonarCloud

Trivy :

Trivy est configuré dans `.github/workflows/trivy-scan.yml`. Il analyse :

- les images Docker (frontend et backend),
- le code source (secrets, fichiers sensibles),
- les `Dockerfiles`,
- les dépendances npm.

Les failles sont classées par gravité (critique, haute, moyenne, faible).

La figure 3.20 montre le résultat de trivy

```
Total: 2 (LOW: 1, MEDIUM: 1, HIGH: 0, CRITICAL: 0)

┌──────────┬────────────────┬──────────┬────────┬───────────────┬───────────────┬──────────────────────────────────────┐
│ Library  │ Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                Title                 │
├──────────┼────────────────┼──────────┼────────┼───────────────┼───────────────┼──────────────────────────────────────┤
│ undici   │ CVE-2025-22150 │ MEDIUM   │ fixed  │ 6.19.7        │ 5.28.5, 6.21.1, 7.2.3 │ undici: Undici Uses Insufficiently Random Values │
│          │                │          │        │               │               │ https://avd.aquasec.com/nvd/cve-2025-22150 │
│          ├────────────────┼──────────┤        │               ├───────────────┼──────────────────────────────────────┤
│          │ CVE-2025-47279 │ LOW      │        │               │ 5.29.0, 6.21.2, 7.5.0 │ undici: Undici Memory Leak with Invalid Certificates │
│          │                │          │        │               │               │ https://avd.aquasec.com/nvd/cve-2025-47279 │
```

Fig. 3.20 – Résultat de Trivy

Les rapports (résumé, Docker, npm) sont stockés comme artefacts.

Cette figure 3.21 présente les résultats de Trivy comme Artefacts dans GitHub Actions

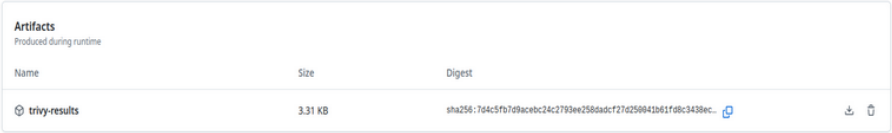

FIG. 3.21 – Résultats de Trivy dans GitHub Actions

Snyk :

Snyk est configuré dans `.github/workflows/snyk-security.yml` et utilise :

— un `SNYK_TOKEN` stocké dans les *secrets*,

— Node.js v18 avec mise en cache npm,

— les options `--all-projects`, `--detection-depth=4`, `--dev`, `--severity-threshold=high`, `--json`.

Les résultats sont visibles sur le dashboard Snyk (`https://snyk.io`) et stockés comme artefacts.

La figure 3.22 montre les résultats de scan dans le dashboard de Snyk :

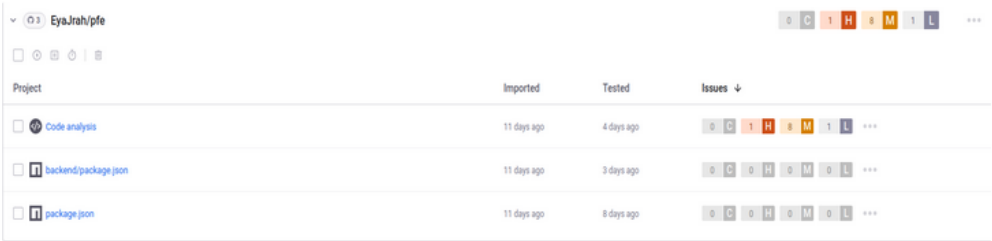

FIG. 3.22 – Dashboard de Snyk

La figure 3.23 montre les résultats de scan de Snyk dans GitHub Actions

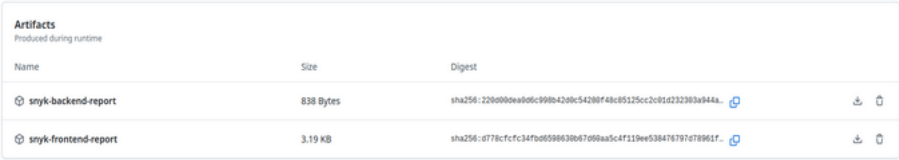

FIG. 3.23 – Résultats de Snyk dans GitHub Actions

OWASP Dependency Check :

OWASP Dependency Check est définit dans `.github/workflows/owasp-check.yml`. Ce workflow effectue :

— une analyse standard (CVSS 7),

— une analyse approfondie (frontend/backend séparés, CVSS 11).

Les résultats sont disponibles en formats HTML et JSON, et enregistrés comme artefacts GitHub. La figure 3.24 représente le résultat de OWASP Dependency-check dans GitHUb Actions

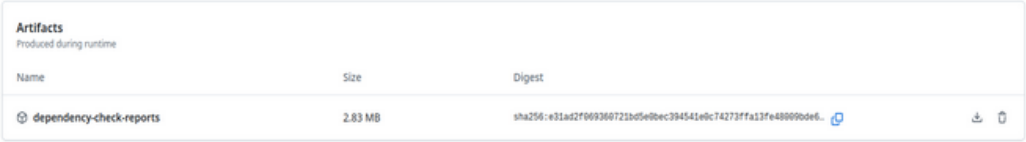

Fig. 3.24 – Résultats de scan de OWASP DEPENDENCY-CHECK dans GitHub Actions

La figure 3.25 montre les résultats de scan de Dependency-check

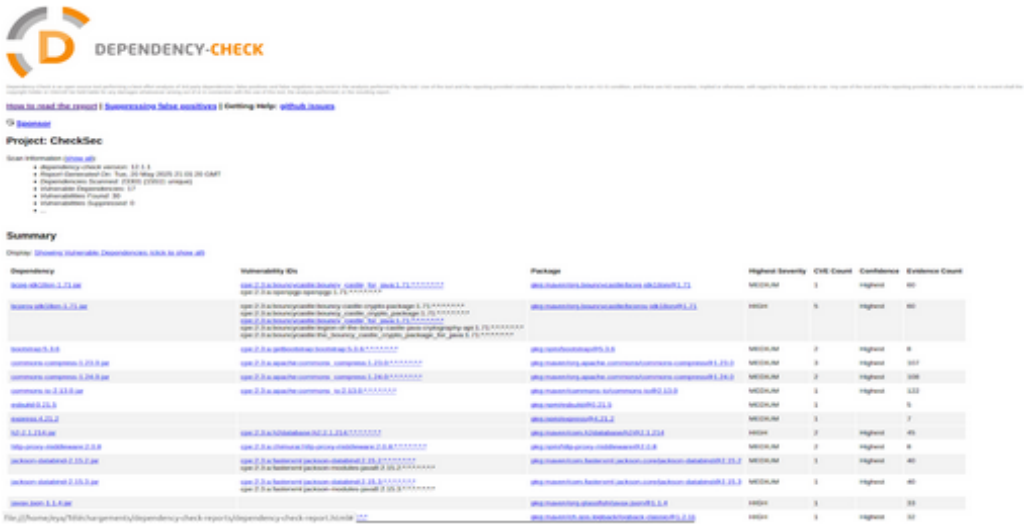

Fig. 3.25 – Résultats de scan de OWASP DEPENDENCY-CHECK

Synthèse :

Ces outils assurent une couverture complète :

— SonarCloud : qualité du code,

— Trivy : sécurité système et conteneurs,

— Snyk : sécurité des dépendances,

— OWASP : vulnérabilités connues des librairies.

3.3 Docker Build and Push :

Ce workflow (`.github/workflows/docker-build-push.yml`) automatise la construction et la publication des images Docker à chaque modification du code. Il :

— récupère le code depuis GitHub,

— configure l'environnement,

— se connecte à Docker Hub,

— construit les images frontend/backend via leurs Dockerfiles,

— publie les images avec des tags versionnés.

La figure 3.26 illustre la configuration des identifiants Docker enregistrés dans GitHub Actions.

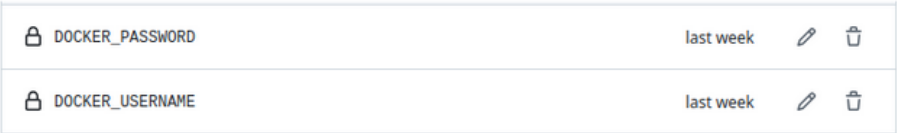

FIG. 3.26 – Identifiants Docker stockés dans GitHub Actions

Ce processus utilise un cache pour optimiser la construction.

Dans cette partie, nous présentons la figure 3.27, qui montre le tableau de bord DockerHub où la dernière version de l'image est automatiquement stockée à chaque push de code.

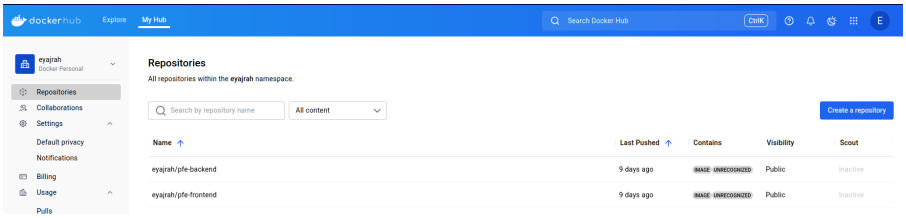

FIG. 3.27 – le dashboard de Docker Hub

Conclusion :

Ce sprint a été marqué par plusieurs avancées majeures, notamment le développement d'une interface Angular moderne, la mise en place d'une architecture backend sécurisée, la configuration d'un pipeline d'intégration continue (CI) complet et sécurisé, ainsi que l'automatisation de la publication des images Docker. Le prochain sprint se concentrera sur la mise en œuvre du déploiement continu (CD) de l'application.

CHAPITRE 4

Sprint 2 : Dockerisation et déploiement de l'application

Sommaire

Introduction 49

4.1 Déploiement Continu 49

4.2 Backlog du sprint 2 49

4.2.1 User Story 07 : Déploiement de CheckSec 50

4.2.2 Configuration Docker 50

4.3 Déploiement avec Render et Railway 52

4.4 Déploiement local avec Docker et Nginx 55

4.4.1 Déploiement local avec Docker et Nginx 55

4.4.2 Déploiement de la partie Frontend 56

4.5 Déploiement de la partie Backend 58

4.6 Orchestration avec Docker Compose 58

4.7 Avantages et limites du déploiement local 59

4.7.1 Avantages 59

4.7.2 Limites 60

Conclusion 60

Introduction

Après avoir mis en place l'intégration continue (CI) lors du Sprint 1 – incluant le développement de l'application, la configuration des workflows GitHub Actions pour automatiser les tests de sécurité et de qualité, et la synchronisation des résultats dans un tableau de bord – ce second sprint est dédié à l'implémentation du déploiement continu (CD).

4.1 Déploiement Continu

Contrairement aux pipelines CI, le déploiement continu n'a pas été intégré dans GitHub Actions. À la place, une stratégie de déploiement directe a été adoptée en utilisant Docker pour la conteneurisation de l'application (frontend et backend) et Render.com comme plateforme de déploiement cloud. Cette approche permet un déploiement automatisé et sécurisé sans nécessiter de configuration complexe dans les workflows GitHub Actions.

La figure 4.1 montre les différentes étapes de pipeline CI/CD

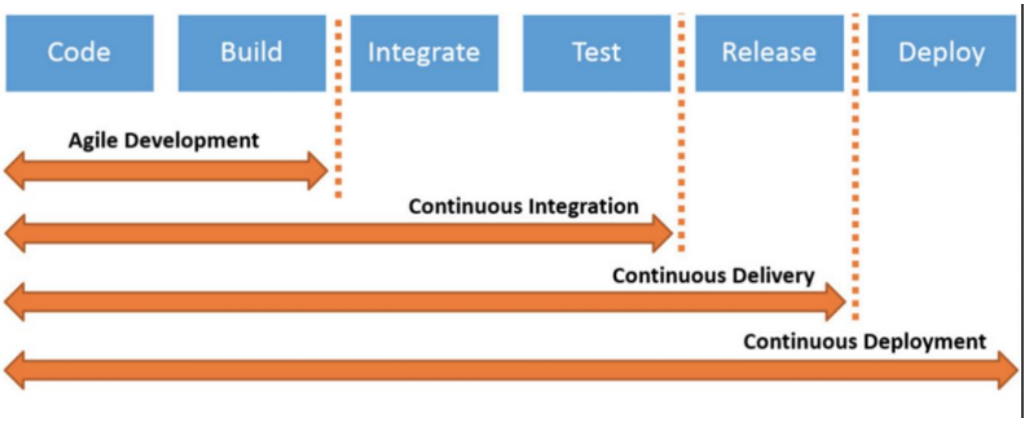

Fig. 4.1 – Pipeline CI/CD

Le Déploiement Continu (CD) est une pratique qui automatise le processus de mise en production des applications. Dans notre cas, il s'agit d'un processus automatisé qui déploie *CheckSec* sur Render.com après chaque validation réussie des tests et des vérifications de sécurité.

4.2 Backlog du sprint 2

Le backlog du deuxième sprint contient une seule User Story, considérée comme essentielle pour le bon déroulement du projet. Elle porte sur la mise en place du déploiement automatique

de l'application. Le tableau 4.1 détaille le User Story. Elle est identifiée par un identifiant unique (ID), décrite en termes de besoin utilisateur, et accompagnée d'une évaluation de priorité (Haute ou Basse)

ID	User Story	Priorité	Complexité
07	En tant qu'utilisateur, je désire le déploiement automatique de mon application	Haute	8

Tab. 4.1 – Backlog du Sprint 2

4.2.1 User Story 07 : Déploiement de CheckSec

Pour le déploiement de *CheckSec*, Docker et la plateforme Render.com ont été utilisés. Docker a permis de créer une image unique contenant le frontend Angular, le backend Node.js, ainsi que les outils de sécurité intégrés (Trivy, Snyk, SonarCloud, Dependency-Check).

Fig. 4.2 – Logo de Docker

Grâce à l'intégration directe de Render avec GitHub, chaque push sur la branche principale déclenche automatiquement la reconstruction et le redéploiement de l'application.

Fig. 4.3 – Logo de Render

4.2.2 Configuration Docker

L'installation de Docker se fait en ligne de commande via le gestionnaire de paquets d'Ubuntu. Le script présenté dans le Listing 4.1 détaille les étapes nécessaires. Il est conseillé de redémarrer la session ou d'exécuter newgrp docker après l'ajout de l'utilisateur au groupe pour que les modifications prennent effet immédiatement.

```
# Met à jour les paquets
sudo apt update

# Installe Docker depuis les dépôts Ubuntu
sudo apt install docker.io -y

# Active le service Docker au démarrage
sudo systemctl enable docker

# Démarre le service Docker maintenant
sudo systemctl start docker

# Ajoute ton utilisateur au groupe docker
sudo usermod -aG docker $USER
```

Listing 4.1 - Installation de Docker

Création de Dockerfile

L'application se compose de deux parties principales : un frontend Angular et un backend Node.js. Le fichier Dockerfile a été conçu de manière à automatiser la construction de l'ensemble du projet dans une seule image Docker cohérente.

Le processus de création du Dockerfile inclut les étapes suivantes :

— Installation des dépendances et compilation du frontend Angular, généralement situé dans un sous-dossier (/frontend). Cette étape génère les fichiers statiques (HTML, CSS, JavaScript) prêts à être servis.

— Installation des dépendances du backend Node.js (situé dans /backend), ainsi que la mise en place de la logique serveur et des API REST.

— Copie des fichiers générés vers un dossier accessible par le serveur, ou intégration dans un serveur léger comme Nginx si le frontend est servi indépendamment.

— Ajout d'outils de scan automatisé, qui sont exécutés au moment de la construction ou du démarrage du conteneur, afin de renforcer la sécurité du code et des dépendances.

4.3 Déploiement avec Render et Railway

Après s'être connecté à la plateforme https://render.com via GitHub et avoir sélectionné le dépôt contenant le projet à déployer (voir figure 4.4), plusieurs étapes sont nécessaires pour configurer un nouveau service web.

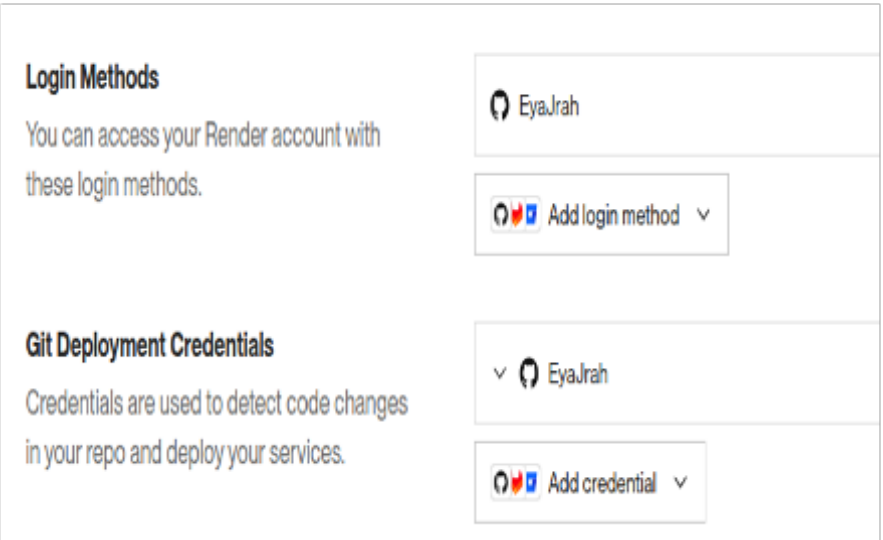

Fig. 4.4 – Formulaire de configuration Render

Tout d'abord, il faut cliquer sur le bouton « New > Web Service » dans le tableau de bord Render afin d'initier la création du service (voir figure 4.5).

Ensuite, l'utilisateur est invité à remplir un formulaire de configuration contenant des informations telles que le nom du service, la branche Git à déployer, le type d'environnement (par exemple Node.js ou Docker), ainsi que la commande de lancement. Ces étapes sont illustrées dans les figures 4.6 et 4.7.

Une fois le service défini, il est essentiel d'ajouter les variables d'environnement nécessaires au bon fonctionnement de l'application. Parmi celles-ci, la plus critique est généralement la chaîne de connexion à la base de données MongoDB Atlas, qui doit être renseignée avec précision pour permettre au backend de se connecter à la base distante (voir figure 4.8).

Cette procédure permet un déploiement rapide et automatisé d'une application web via GitHub, tout en assurant la personnalisation nécessaire pour chaque environnement de production.

— Cliquer sur "New > Web Service" (voir figure 4.5).

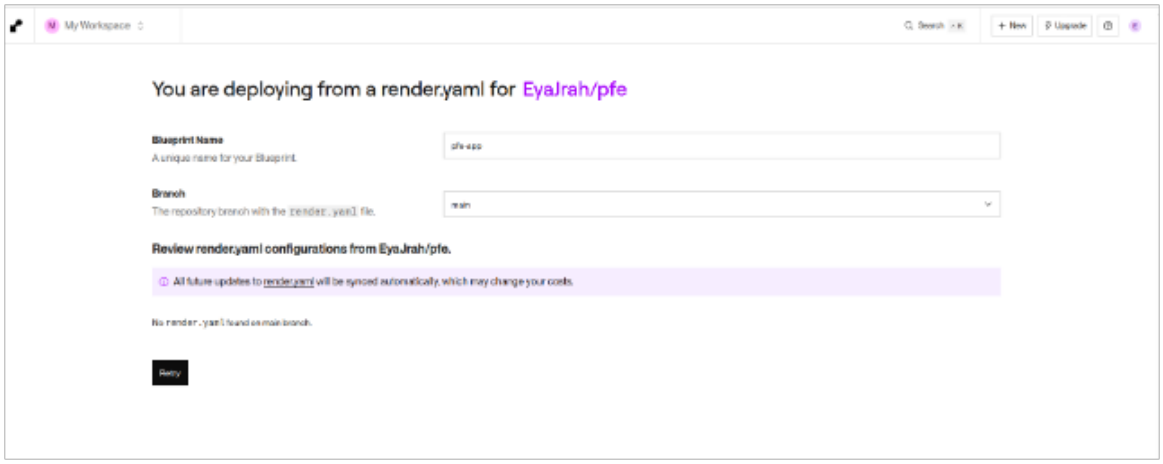

FIG. 4.5 – Création de nouveau service web

— Remplir le formulaire de configuration (voir figures 4.6 et 4.7).

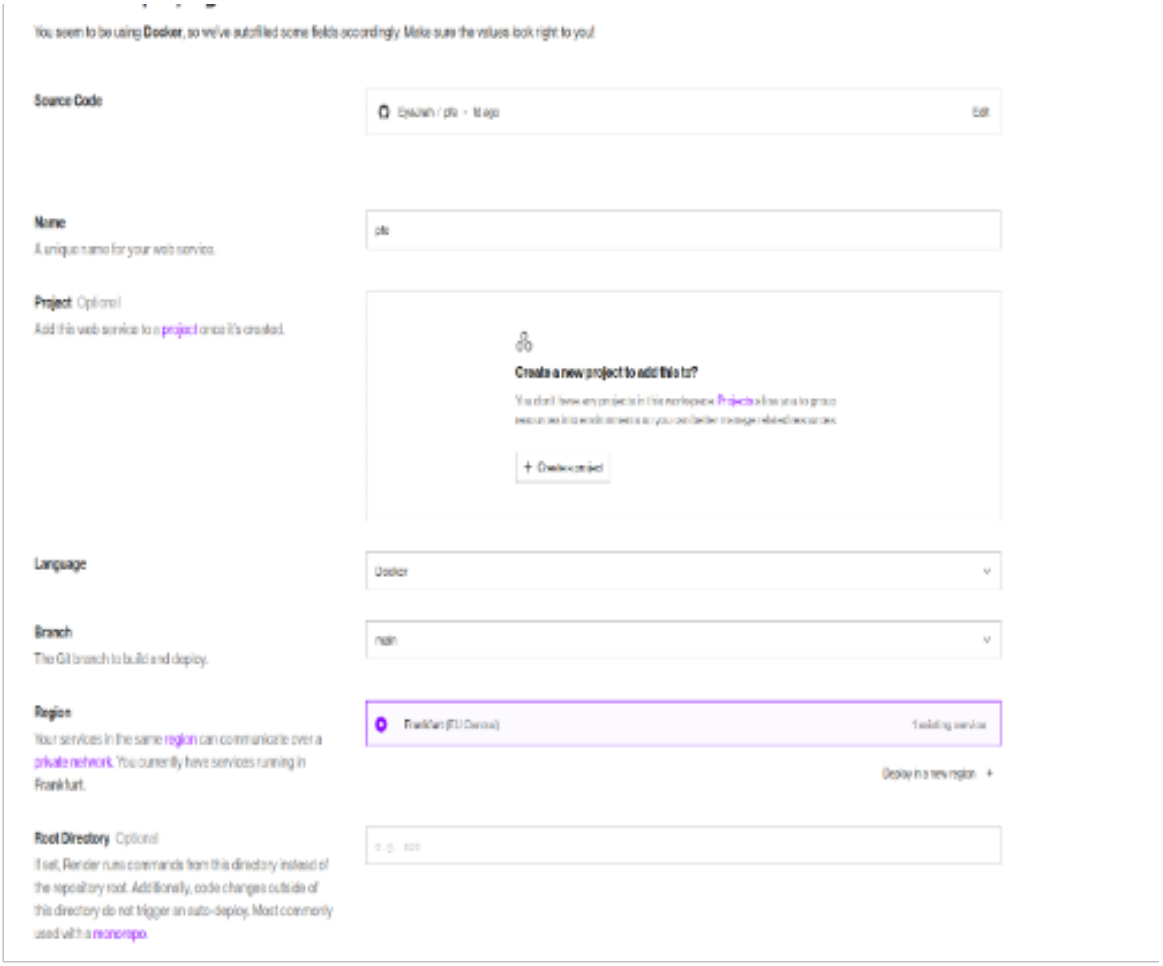

FIG. 4.6 – Formulaire de configuration de service web 1

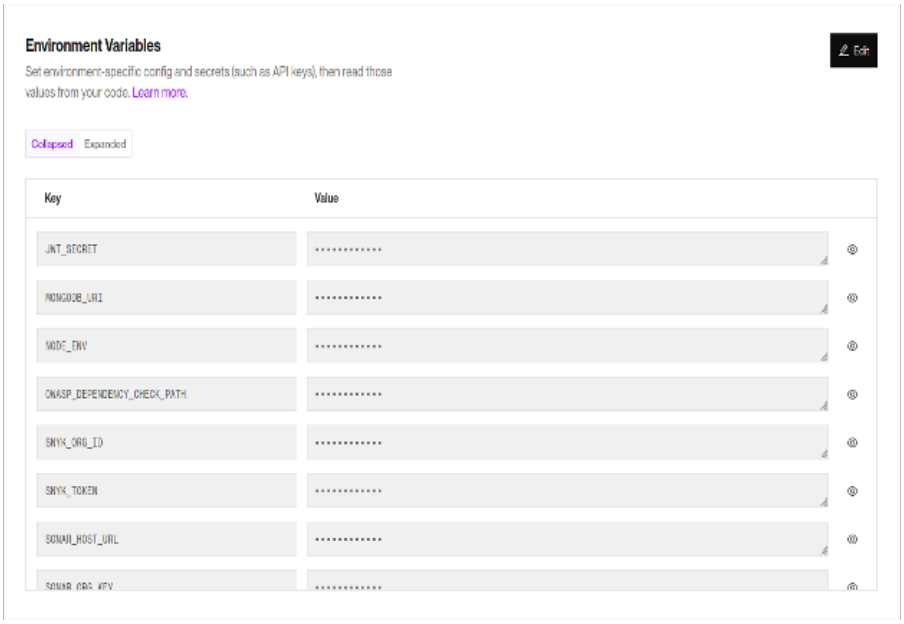

FIG. 4.7 – Formulaire de configuration de service web 2

— Définir les variables d'environnement, notamment la chaîne de connexion MongoDB Atlas, comme le montre la figure 4.8.

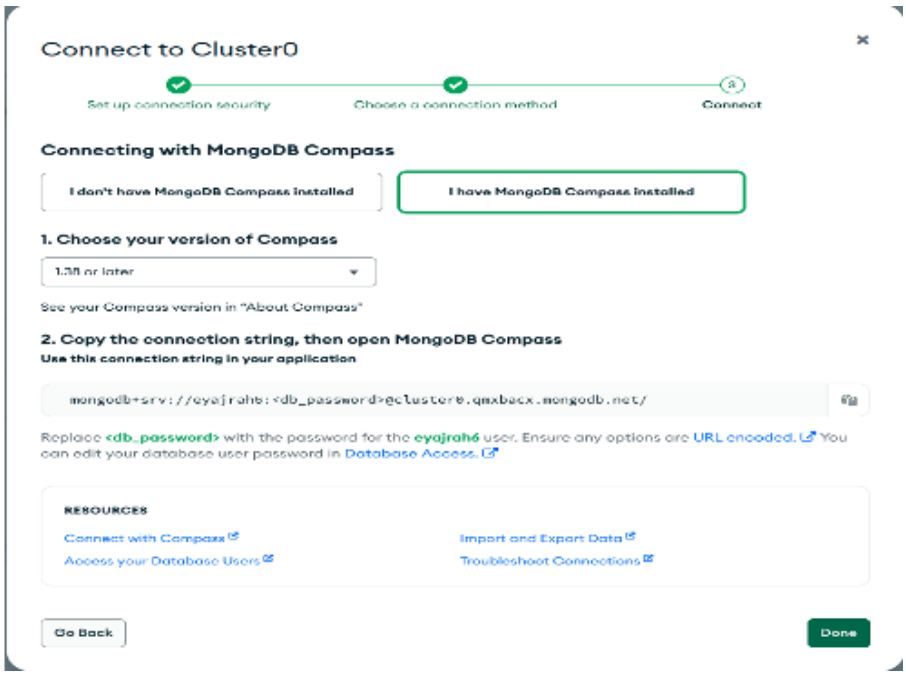

FIG. 4.8 – Connexion de base de données MongoDB Atlas

Nous avons utilisé une base de données MongoDB Atlas qui est une base de données dans le cloud. Elle nous permet de stocker, gérer et interroger les données de manière sécurisée et évolutive. Grâce à son interface conviviale et sa compatibilité avec MongoDB Compass, elle facilite le suivi et l'administration des collections de données utilisées par l'application. Nous nous sommes connectés à la base de données via Render en ajoutant la chaîne de connexion

dans le champ des variables d'environnement.

L'application a été correctement déployée et connectée à la base de données. Cependant, la fonctionnalité de scan ne fonctionne pas en raison d'un problème de timeout sur Render : ce délai est trop court par rapport au temps nécessaire au scan. Nous avons donc décidé de migrer vers Railway, une plateforme cloud similaire à Render.

Nous avons utilisé le même Dockerfile avec un service web et ajouté les variables d'environnement, en répétant les mêmes étapes de configuration sur Railway. La figure 4.9 illustre la configuration et le tableau de bord de Railway.

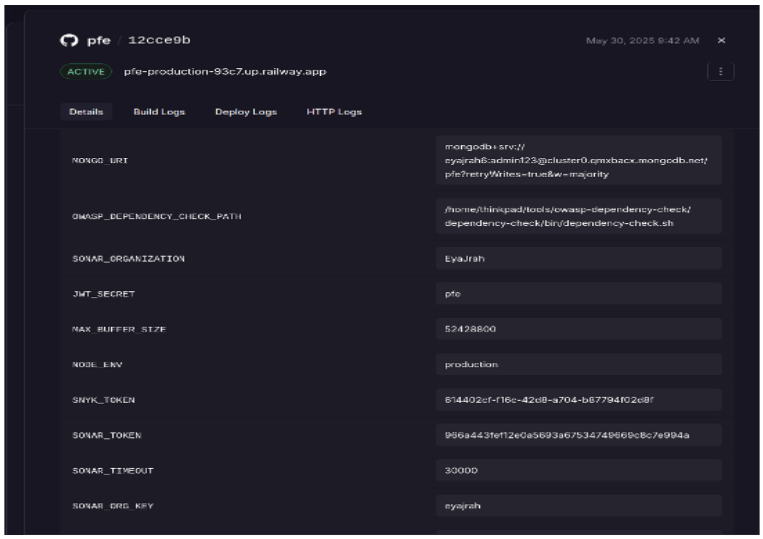

Fig. 4.9 – Configuration et tableau de bord de Railway

4.4 Déploiement local avec Docker et Nginx

4.4.1 Déploiement local avec Docker et Nginx

Pour assurer un déploiement efficace, portable et reproductible de l'application, plusieurs outils et technologies ont été mis en œuvre (voir la figure 4.10).

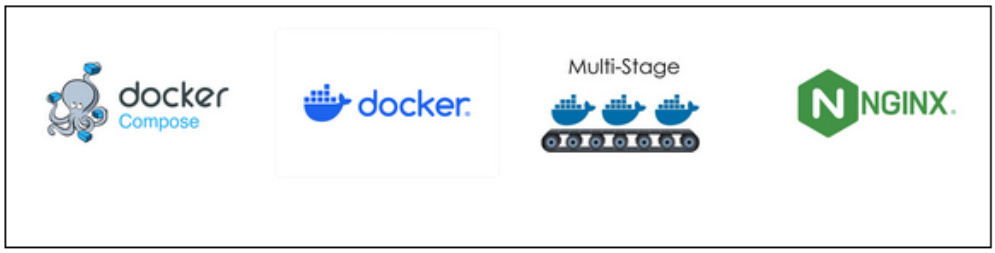

Fig. 4.10 – Les outils de déploiement local

Docker est utilisé pour la création d'images et l'isolation des environnements d'exécution,

permettant ainsi de garantir que l'application fonctionne de manière identique sur toute machine compatible. Docker Compose facilite l'orchestration et la gestion multi-conteneurs, en automatisant le lancement coordonné du frontend, du backend et du serveur web. Le multi-stage build dans les Dockerfiles permet d'optimiser la taille des images en ne conservant que les fichiers nécessaires à l'exécution finale. Nginx est utilisé comme serveur web performant pour servir les fichiers statiques du frontend et, si besoin, comme reverse proxy pour le backend, assurant ainsi rapidité, sécurité et modularité à l'architecture globale.

4.4.2 Déploiement de la partie Frontend

Le déploiement du frontend repose sur l'utilisation d'un multi-stage build dans le Dockerfile : une technique qui permet d'optimiser la taille de l'image finale et de garantir qu'elle ne contient que les fichiers strictement nécessaires à l'exécution de l'application.

Le déploiement en utilisant un Dockerfile avec *multi-stage build* :

— Étape 1 : node :18-alpine pour build Angular

— Étape 2 : nginx :alpine pour servir les fichiers statiques

La figure 4.11 montre le dockerfile de frontend :

```
# Stage 1: Build the React app
FROM node:18-alpine AS builder
WORKDIR /app
COPY package.json ./
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Serve with Nginx
FROM nginx:alpine
RUN rm -rf /usr/share/nginx/html/*
COPY --from=builder /app/dist/temp-app/browser /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Fig. 4.11 – Dockerfile du frontend

Dans une première étape, une image Node.js (node :18-alpine) est utilisée pour installer les dépendances et construire l'application frontend. Le code source et les fichiers de configuration sont copiés dans l'image, puis la commande npm run build génère les fichiers statiques de production.

Dans une seconde étape, une image nginx légère (nginx :alpine) est utilisée. Les fichiers statiques générés lors de l'étape précédente sont copiés dans le répertoire /usr/share/nginx/html

de l'image nginx. Cette approche permet de bénéficier des performances et de la robustesse de nginx pour servir les fichiers du frontend. Grâce à cette méthode, l'image finale du frontend est à la fois légère, rapide à déployer et adaptée à un environnement de production. L'utilisation de nginx garantit également une gestion efficace du cache, de la compression et de la distribution des fichiers statiques auprès des utilisateurs.

En plus de servir les fichiers statiques, Nginx peut également agir comme reverse proxy. Cela signifie qu'il peut rediriger certaines requêtes, par exemple celles destinées à l'API (comme /api), vers le serveur backend Node.js. Cette configuration permet de centraliser les accès via un seul point d'entrée (Nginx). La figure 4.12 représente le fichier de configuration Nginx.

```
events {}

http {
  server {
    listen 80;
    server_name localhost;

    root /usr/share/nginx/html;
    index index.html index.htm;

    location / {
      try_files $uri $uri/ /index.html;
      include /etc/nginx/mime.types;
    }

    # Proxy pour le backend (API)
    location /api/ {
      proxy_pass http://backend:5000/;
      proxy_http_version 1.1;
      proxy_set_header Upgrade $http_upgrade;
      proxy_set_header Connection 'upgrade';
      proxy_set_header Host $host;
      proxy_cache_bypass $http_upgrade;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
  }
}
```

Fig. 4.12 – Fichier de configuration Nginx

Pour l'accès via un nom de domaine local, un nom de domaine personnalisé tel que `www.checksec.loc` a été utilisé. Pour que ce nom de domaine fonctionne en local, il est nécessaire d'ajouter une entrée dans le fichier `/etc/hosts` de la machine.

La figure 4.13 montre le fichier /etc/hosts après l'ajout de domaine personnalisé

```
  GNU nano 7.2                          hosts
127.0.0.1 localhost
127.0.1.1 eya-ThinkPad-E16-Gen-1

# The following lines are desirable for IPv6 capable hosts
::1     ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
127.0.0.1  www.checksec.loc
```

Fig. 4.13 – Entrée dans le fichier `/etc/hosts`

4.5 Déploiement de la partie Backend

Le déploiement du backend s'appuie sur un Dockerfile dédié, conçu pour garantir à la fois la portabilité, la sécurité et la reproductibilité de l'environnement d'exécution. L'image Docker du backend est construite à partir d'une base openjdk :21-jdk-bullseye, ce qui permet d'intégrer à la fois Node.js et Java, nécessaires au fonctionnement de l'application et des outils d'analyse de sécurité.

Le Dockerfile backend configure l'environnement Node.js pour l'application Express en installant les dépendances via npm install et en intégrant des outils de sécurité (Trivy, Snyk, OWASP Dependency-check et SonarCloud). Il prépare l'environnement en créant les dossiers nécessaires, attribuant les droits d'exécution aux scripts et configurant les variables d'environnement, puis lance simultanément le script d'analyse de sécurité et le serveur Node.js au démarrage du conteneur.

La figure 4.14 représente le dockerfile créé pour le Backend de L'application

```
FROM openjdk:21-jdk-bullseye
# ... (installation de Node.js, outils, dépendances)
WORKDIR /app
COPY package*.json ./
RUN npm install && npm cache clean --force
COPY . .
RUN chmod +x scan-and-send.sh setup-sonar.sh setup-trivy.sh
RUN mkdir -p /tmp/dependency-check/dc-report
EXPOSE 5000
CMD ["/bin/sh", "-c", "cd /app && /app/scan-and-send.sh & node server.js"]
```

Fig. 4.14 – Dockerfile du backend

4.6 Orchestration avec Docker Compose

L'orchestration de l'ensemble des services de l'application est assurée par Docker Compose, via le fichier docker-compose.yaml. Cet outil permet de définir, configurer et lancer plusieurs conteneurs Docker de manière coordonnée, simplifiant ainsi le déploiement et la gestion de l'architecture multi-composants.

L'application s'articule autour de trois services principaux : backend (API et sécurité), frontend (interface utilisateur) et nginx (serveur web) orchestrés automatiquement par Docker Compose dans des conteneurs isolés. Ces services communiquent via un réseau Docker privé sécurisé checksec-net, partagent des fichiers essentiels grâce aux volumes et s'adaptent facilement aux différents environnements via les variables d'environnement. Cette architecture conteneu-

risée assure un déploiement simple, reproductible et flexible tout en maintenant la cohérence et la sécurité de l'ensemble du système. En résumé, il s'agit de :

— Trois services : frontend, backend, nginx

— Réseau privé sécurisé `checksec-net`

— Partage de volumes et utilisation de variables d'environnement

Dans cette section , La figure 4.15 présente le docker-compose

```yaml
services:
  backend:
    build:
      context: ./backend
    container_name: checksec-backend
    ports:
      - "5000:5000"
    volumes:
      - ./backend:/app
      - /tmp/dependency-check:/tmp/dependency-check
    environment:
      - MONGODB_URI=...
      # autres variables

  frontend:
    build:
      context: ./frontend
    container_name: checksec-front
    expose:
      - "80"

  nginx:
    image: nginx:alpine
    container_name: checksec-nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - ./frontend/dist/temp-app/browser:/usr/share/nginx/html:ro
    depends_on:
      - frontend
      - backend

networks:
  checksec-net:
    driver: bridge
```

FIG. 4.15 – Fichier `docker-compose.yaml`

4.7 Avantages et limites du déploiement local

4.7.1 Avantages

Le choix d'un déploiement local présente plusieurs atouts significatifs :

— **Coût réduit** : aucun frais d'hébergement cloud

— **Sécurité** : les données restent sur l'infrastructure locale

— **Simplicité** : déploiement avec une seule commande

4.7.2 Limites

Malgré ses avantages, le déploiement local comporte aussi certaines contraintes :

— **Accès limité** : uniquement depuis la machine locale

— **Dépendance locale** : la stabilité repose sur l'environnement de la machine

Conclusion

En résumé, le choix d'une architecture basée sur Docker et Docker Compose a permis un déploiement efficace, reproductible et conforme aux principes DevOps. Bien que la solution cloud ait présenté des limitations liées aux essais gratuits et aux performances, le déploiement local a permis de garantir la continuité du projet jusqu'à la soutenance.

Conclusion Générale

Ce projet s'inscrit dans une démarche globale de renforcement de la sécurité des applications web, tout en modernisant les pratiques de développement et de déploiement. L'objectif principal était d'intégrer l'automatisation des tests de sécurité dès les premières étapes du cycle de vie logiciel (SDLC), à travers la mise en place d'une chaîne CI/CD enrichie par des outils de scan adaptés, et d'assurer un déploiement cohérent et sécurisé.

Le travail a été mené selon une approche agile, fondée sur la méthodologie Scrum, avec une organisation en sprints successifs permettant une itération rapide et des résultats tangibles à chaque étape. Dès le début, une phase de planification a permis de définir les besoins et les priorités, avant de procéder à l'intégration d'outils tels que SonarCloud (analyse statique de code), Trivy (scan d'images Docker), Snyk et OWASP Dependency Check (vérification des dépendances). Par ailleurs, une interface web développée en Angular permet de visualiser les résultats des différents tests, tandis qu'un backend en Node.js avec MongoDB assure la centralisation et la gestion des données de sécurité.

La containerisation de l'ensemble à l'aide de Docker, couplée à un déploiement automatisé via Docker Compose, a permis de créer un environnement local reproductible et sécurisé, illustrant concrètement les principes DevSecOps, où la sécurité devient une responsabilité partagée dès les premières phases du développement.

Ce travail a constitué une expérience formatrice et enrichissante, tant sur le plan technique que professionnel. Il a permis de renforcer les compétences en cybersécurité, en développement web moderne, en intégration continue et en automatisation. Il a également été l'occasion d'expérimenter la gestion d'un projet complexe sous contraintes de temps, de qualité et de performance.

En conclusion, ce projet a atteint ses objectifs en proposant une solution concrète, efficace et évolutive pour l'automatisation des tests de sécurité dans les applications web. Il témoigne d'une prise de conscience des enjeux liés à la sécurité logicielle.

Bibliographie

[1] Gene Kim, Patrick Debois, John Willis, and Jez Humble. *The DevOps Handbook : How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, 2016.

[2] Ian Miell and Aidan Hobson Sayers. *Docker in Practice*. Manning Publications Co., USA, 1st edition, 2016.

[3] Véronique Messager Rota. *Gestion de projet agile : avec Scrum, Lean, Extreme Programming...* Génie logiciel. Dunod, Paris, 2020.

[4] Ken Schwaber and Mike Beedle. *Agile Software Development with Scrum*. Prentice Hall, Upper Saddle River, New Jersey, 2002.